TRIALGO

JEU DE CARTES · APPLICATION MOBILE

RECUEIL COMPLET DE CONCEPTION — VERSION 3.0 MODÈLES DE DONNÉES · IMAGES · CHEMINS STORAGE · MÉCANIQUE DE JEU

Rôle	Responsable
Chef de projet	FOKAM FEKAM Cédrick
Chef technique	TCHAMBA TCHAKOUNTE Edwin
Chef artistique	HAPPE WAKAM Loïc Belmond
Stack technique	Flutter · Supabase · Riverpod · Clean Architecture
Version	v3.0 — Révision majeure : Câble = Image + chemins Storage

1. PRINCIPE FONDAMENTAL — LES TROIS CARTES SONT TROIS IMAGES

Point de départ absolu : dans TRIALGO, toutes les cartes sans exception sont des illustrations dessinées et stockées en tant qu'images. Il n'existe aucun algorithme abstrait ou textuel. La carte Câble, que l'on pourrait appeler "carte de transformation", n'est pas une règle écrite — c'est elle-même une image dessinée qui représente visuellement le type de transformation qu'elle encode.

RÉVISION FONDAMENTALE — Version 3.0

ANCIENNE CONCEPTION (incorrecte) :
Émettrice = image + Câble = règle abstraite (code texte) + Réceptrice = image

NOUVELLE CONCEPTION (correcte) :
Émettrice = IMAGE + Câble = IMAGE + Réceptrice = IMAGE

Les trois cartes sont trois illustrations physiques dessinées par le chef artistique.
Chacune a son propre fichier image, son propre chemin dans Supabase Storage,
sa propre URL publique. La carte Câble encode visuellement sa transformation.

Conséquence directe : la table "algorithms" abstraite est SUPPRIMÉE.
La logique de transformation est entièrement encodée dans l'image de la carte Câble.

1.1 Ce que représente visuellement chaque type de carte

Type de carte	Ce qu'elle représente visuellement	Exemple concret	Peut être la carte demandée ?
Émettrice (E)	Une illustration de base : un objet, un animal, une scène	Dessin d'un lion assis, couleurs vives	OUI — niveau avancé (config C)
Câble (C)	Une illustration qui représente visuellement une transformation : flèche de miroir, cercles de couleur, icône de rotation, motif de fragmentation...	Dessin de deux flèches se faisant face (= miroir horizontal)	OUI — niveau intermédiaire (config B)
Réceptrice (R)	L'illustration résultante : l'émettrice après que la transformation du câble lui a été appliquée	Lion miroir (reflet du lion de base)	OUI — niveau débutant (config A, par défaut)

Règle visuelle de la carte Câble

La carte Câble doit être LISIBLE visuellement comme une transformation.
Le joueur doit pouvoir comprendre, en regardant l'image du Câble,
quel type d'opération visuelle il représente.

Exemples de représentations visuelles de câbles :

- Deux flèches symétriques → représente le miroir horizontal
- Une palette de couleurs rouges → représente la teinte rouge
- Un cercle divisé en 3 parts → représente la fragmentation en 3
- Une loupe avec "×2" → représente l'agrandissement ×2
- Une roue tournante à 90° → représente la rotation 90°

Le dessin du Câble EST l'algorithme. Pas de code texte, pas de règle cachée.

1.2 La fusion de deux images pour produire une troisième

La relation entre les trois cartes est toujours une fusion visuelle : l'image de l'Émettrice, interprétée à travers la règle visuelle que porte l'image du Câble, produit l'image de la Réceptrice. C'est une opération à trois images.

Formule révisée — Fusion de deux images

IMAGE Émettrice ⊕IMAGE Câble = IMAGE Réceptrice

"Lion de base" ⊕"Flèches miroir" = "Lion miroir"

Les trois ont leur propre fichier, leur propre chemin Storage, leur propre URL.
Aucune des trois n'est générée automatiquement par code — elles sont toutes dessinées manuellement par le chef artistique et uploadées dans Supabase Storage.

1.3 Le système de distances — inchangé dans la logique, enrichi dans la représentation

Le principe de distances reste le même : une chaîne de fusions successives. Ce qui change : à chaque niveau, il y a un nouveau Câble-image ET une nouvelle Réceptrice-image. Tout est dessiné, rien n'est calculé automatiquement.

Distance	Fusion	Formation	Nombre d'images à dessiner	Exemple
D1	E1 ⊕C1 = R1	Trio (3 cartes)	3 images distinctes	Lion + [Flèches miroir] = Lion miroir
D2	R1 ⊕C2 = R2 (R1 joue le rôle d'E2)	Quintette (5 cartes)	+2 images (C2 + R2)	Lion miroir + [Palette rouge] = Lion miroir rouge
D3	R2 ⊕C3 = R3 (R2 joue le rôle d'E3)	Septette (7 cartes)	+2 images (C3 + R3)	Lion miroir rouge + [Cercle en 3] = Lion miroir rouge fragmenté

2. ORGANISATION DES FICHIERS — SUPABASE STORAGE

Puisque toutes les cartes sont des images, l'organisation du stockage est critique. Supabase Storage est utilisé comme CDN d'images. Chaque image a un chemin structuré et une URL publique stable.

2.1 Structure des buckets Supabase Storage

Bucket principal : trialgo-cards (public)

```
trialgo-cards/
├── emettrices/
│   ├── savane/
│   │   ├── lion_base.webp
│   │   ├── elephant_base.webp
│   │   └── girafe_base.webp
│   ├── ocean/
│   │   ├── requin_base.webp
│   │   └── baleine_base.webp
│   └── foret/
│       └── renard_base.webp
│
├── cables/
│   ├── geometrique/
│   │   ├── miroir_h.webp
│   │   ├── miroir_v.webp
│   │   ├── rotation_90.webp
│   │   └── rotation_180.webp
│   ├── couleur/
│   │   ├── teinte_rouge.webp
│   │   ├── teinte_bleue.webp
│   │   ├── niveaux_gris.webp
│   │   └── inversion.webp
│   ├── dimension/
│   │   ├── agrandissement_2x.webp
│   │   └── reduction_2x.webp
│   └── complexe/
│       ├── fragmentation_3.webp
│       └── ombre_portee.webp
│
└── receptrices/
    ├── savane/
    │   ├── d1/
    │   │   ├── lion_miroir_h.webp
    │   │   ├── lion_teinte_rouge.webp
```

```
    │   │   └── lion_rotation_90.webp
    │   ├── d2/
    │   │   ├── lion_miroir_h_teinte_rouge.webp
    │   │   └── lion_miroir_h_rotation_90.webp
    │   └── d3/
    │       └── lion_miroir_h_teinte_rouge_fragmente.webp
    └── ocean/
        └── d1/
            └── requin_miroir_h.webp
```

2.2 Convention de nommage des fichiers

Une convention stricte garantit la lisibilité et évite les collisions de noms.

Type de carte	Convention	Exemple
Émettrice	{theme}/{sujet}_base.webp	savane/lion_base.webp
Câble	{categorie}/{transformation}.webp	geometrique/miroir_h.webp
Réceptrice D1	{theme}/d1/{sujet}_{transformation}.webp	savane/d1/lion_miroir_h.webp
Réceptrice D2	{theme}/d2/{sujet}_{transfo1}_{transfo2}.webp	savane/d2/lion_miroir_h_teinte_rouge.webp
Réceptrice D3	{theme}/d3/{sujet}_{t1}_{t2}_{t3}.webp	savane/d3/lion_miroir_h_teinte_rouge_fragmente.webp

Règles de nommage

- Tout en minuscules, tirets bas uniquement (pas d'espaces ni de tirets)
- Extension .webp recommandée (meilleur rapport qualité/taille pour le mobile)
- Ne jamais utiliser d'accents dans les noms de fichiers
- La partie {sujet} de la réceptrice reprend exactement le nom de l'émettrice sans "_base"
- L'ordre des transformations dans le nom de la réceptrice suit l'ordre chronologique de la chaîne

2.3 Construction des URL publiques Supabase

Toutes les images sont dans un bucket public. L'URL publique suit un format stable que l'application peut reconstruire sans requête préalable.

```
// Format de l'URL publique Supabase Storage
https://<PROJECT_REF>.supabase.co/storage/v1/object/public/trialgo-cards/<path>

// Exemples concrets
```

```
https://abcdefgh.supabase.co/storage/v1/object/public/trialgo-
cards/emettrices/savane/lion_base.webp
https://abcdefgh.supabase.co/storage/v1/object/public/trialgo-
cards/cables/geometrique/miroir_h.webp
https://abcdefgh.supabase.co/storage/v1/object/public/trialgo-
cards/receptrices/savane/d1/lion_miroir_h.webp

// DART — Constante de base URL (lib/core/constants/storage_constants.dart)
class StorageConstants {
  static const String baseUrl =
    'https://<PROJECT_REF>.supabase.co/storage/v1/object/public/trialgo-cards';

  static String emettriceUrl(String theme, String filename) =>
    '$baseUrl/emettrices/$theme/$filename';

  static String cableUrl(String category, String filename) =>
    '$baseUrl/cables/$category/$filename';

  static String receptriceUrl(String theme, int distance, String filename) =>
    '$baseUrl/receptrices/$theme/d$distance/$filename';
}
```

3. MODÈLES DE DONNÉES — SCHÉMA SUPABASE RÉVISÉ

La révision fondamentale supprime la table algorithms abstraite. Toutes les cartes (Émettrices, Câbles, Réceptrices) sont des entrées dans la même table cards. Chaque entrée pointe vers son image via un chemin Storage structuré.

3.1 Table : cards — unique table pour les trois types d'images

Une seule table pour les trois types de cartes. La colonne card_type distingue Émettrice, Câble et Réceptrice. La colonne image_path stocke le chemin relatif dans Supabase Storage (pas l'URL complète, pour permettre de changer de projet sans tout migrer).

```
-- SUPABASE / POSTGRESQL — Version 3.0
-- SUPPRESSION de la table algorithms : le câble EST une image, pas un code texte

CREATE TABLE cards (
  id               UUID    PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Type et niveau
  card_type        TEXT    NOT NULL CHECK (card_type IN ('emettrice', 'cable',
'receptrice')),
  distance_level   INT     NOT NULL DEFAULT 1 CHECK (distance_level BETWEEN 1 AND
3),

  -- IMAGE : chemin relatif dans le bucket Supabase Storage "trialgo-cards"
  -- Exemple : "emettrices/savane/lion_base.webp"
  -- Exemple : "cables/geometrique/miroir_h.webp"
  -- Exemple : "receptrices/savane/d1/lion_miroir_h.webp"
  image_path       TEXT    NOT NULL,

  -- Métadonnées de l'image
  image_width      INT,                    -- largeur en pixels (utile pour
l'affichage Flutter)
  image_height     INT,                    -- hauteur en pixels
  image_format     TEXT    DEFAULT 'webp', -- webp | png | jpg

  -- Catégorie et thème (pour les câbles : geometrique | couleur | dimension |
complexe | thematique)
  cable_category   TEXT,                   -- non null uniquement pour les câbles
  theme_tags       TEXT[]  DEFAULT '{}'::TEXT[],

  -- Relations entre cartes (pour les Réceptrices uniquement)
  -- parent_emettrice_id : l'Émettrice (ou Réceptrice jouant le rôle d'Émettrice)
de ce trio
  -- parent_cable_id     : le Câble-image utilisé dans ce trio
  -- root_emettrice_id   : l'Émettrice racine de toute la chaîne (toujours la même
pour D1/D2/D3)
  parent_emettrice_id  UUID REFERENCES cards(id),  -- null pour Émettrices et
Câbles
  parent_cable_id      UUID REFERENCES cards(id),  -- null pour Émettrices et
Câbles
  root_emettrice_id    UUID REFERENCES cards(id),  -- null pour D1, pointe vers E1
pour D2/D3
```

```
  -- Métadonnées de jeu
  difficulty_score FLOAT   DEFAULT 0.5,   -- 0.0 (trivial) à 1.0 (expert)
  is_active        BOOLEAN DEFAULT TRUE,
  created_at       TIMESTAMPTZ DEFAULT now()
);

-- INDEX CRITIQUES
CREATE INDEX idx_cards_type       ON cards(card_type);
CREATE INDEX idx_cards_distance   ON cards(distance_level);
CREATE INDEX idx_cards_parent_e   ON cards(parent_emettrice_id);
CREATE INDEX idx_cards_parent_c   ON cards(parent_cable_id);
CREATE INDEX idx_cards_root       ON cards(root_emettrice_id);
CREATE INDEX idx_cards_tags       ON cards(theme_tags) USING GIN;
CREATE INDEX idx_cards_cable_cat  ON cards(cable_category);
```

Explication des colonnes clés

Colonne	Type	Qui l'utilise	Description
image_path	TEXT NOT NULL	Toutes les cartes	Chemin relatif dans le bucket Storage. Ex: "cables/geometrique/miroir_h.webp". L'URL complète est reconstruite côté client.
cable_category	TEXT	Câbles uniquement	Catégorie visuelle du câble : geometrique | couleur | dimension | complexe | thematique. Permet de filtrer les câbles par type dans la ScrollView.
parent_emettrice_id	UUID	Réceptrices uniquement	Référence la carte qui joue le rôle d'Émettrice dans ce trio. Pour D1 : pointe vers l'Émettrice racine. Pour D2 : pointe vers la Réceptrice D1.
parent_cable_id	UUID	Réceptrices uniquement	Référence l'image de la carte Câble utilisée pour produire cette Réceptrice. C'est une image, pas un code.
root_emettrice_id	UUID	Réceptrices D2, D3	Pointe toujours vers l'Émettrice racine originale (D1). Permet de remonter toute la chaîne rapidement.

3.2 Classe Dart — CardEntity et CardModel

```
// lib/domain/entities/card_entity.dart

enum CardType { emettrice, cable, receptrice }

class CardEntity {
  final String id;
  final CardType cardType;
  final int distanceLevel;
```

```
  // Chemin relatif dans Supabase Storage
  // Ex: "emettrices/savane/lion_base.webp"
  // Ex: "cables/geometrique/miroir_h.webp"
  // Ex: "receptrices/savane/d1/lion_miroir_h.webp"
  final String imagePath;

  // Métadonnées image
  final int? imageWidth;
  final int? imageHeight;
  final String imageFormat;

  // Catégorie (câbles uniquement)
  final String? cableCategory;

  // Tags thématiques
  final List<String> themeTags;

  // Relations (réceptrices uniquement)
  final String? parentEmettriceId;  // Émettrice ou Réceptrice parente
  final String? parentCableId;      // Câble-image utilisé
  final String? rootEmettriceId;    // Émettrice racine de la chaîne

  final double difficultyScore;
  final bool isActive;

  const CardEntity({
    required this.id,
    required this.cardType,
    required this.distanceLevel,
    required this.imagePath,
    this.imageWidth,
    this.imageHeight,
    this.imageFormat = 'webp',
    this.cableCategory,
    this.themeTags = const [],
    this.parentEmettriceId,
    this.parentCableId,
    this.rootEmettriceId,
    this.difficultyScore = 0.5,
    this.isActive = true,
  });

  // Construit l'URL publique complète à partir du chemin relatif
  String get imageUrl => StorageConstants.baseUrl + '/$imagePath';

  // Vérifie si c'est une Émettrice racine (aucun parent)
  bool get isRootEmettrice =>
    cardType == CardType.emettrice && parentEmettriceId == null;

  // Vérifie si c'est un Câble (image de transformation)
  bool get isCable => cardType == CardType.cable;
}
```

```dart
// lib/data/models/card_model.dart

class CardModel extends CardEntity {
  const CardModel({required super.id, required super.cardType,
    required super.distanceLevel, required super.imagePath,
    super.imageWidth, super.imageHeight, super.imageFormat,
    super.cableCategory, super.themeTags, super.parentEmettriceId,
    super.parentCableId, super.rootEmettriceId,
    super.difficultyScore, super.isActive,
  });

  factory CardModel.fromJson(Map<String, dynamic> json) => CardModel(
    id:                  json['id'],
    cardType:            CardType.values.byName(json['card_type']),
    distanceLevel:       json['distance_level'],
    imagePath:           json['image_path'],     // chemin relatif Storage
    imageWidth:          json['image_width'],
    imageHeight:         json['image_height'],
    imageFormat:         json['image_format'] ?? 'webp',
    cableCategory:       json['cable_category'],
    themeTags:           List<String>.from(json['theme_tags'] ?? []),
    parentEmettriceId:   json['parent_emettrice_id'],
    parentCableId:       json['parent_cable_id'],
    rootEmettriceId:     json['root_emettrice_id'],
    difficultyScore:     (json['difficulty_score'] ?? 0.5).toDouble(),
    isActive:            json['is_active'] ?? true,
  );
}
```

3.3 Table : card_trios — relation entre trois images

La table card_trios matérialise chaque combinaison valide de trois images. C'est ici que la relation "Émettrice-image + Câble-image = Réceptrice-image" est officiellement enregistrée.

```sql
-- SUPABASE / POSTGRESQL
CREATE TABLE card_trios (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  -- Les trois images du trio
  emettrice_id     UUID NOT NULL REFERENCES cards(id),   -- image Émettrice
  cable_id         UUID NOT NULL REFERENCES cards(id),   -- image Câble
  receptrice_id    UUID NOT NULL REFERENCES cards(id),   -- image Réceptrice
(résultat)

  distance_level   INT  NOT NULL DEFAULT 1,
  parent_trio_id   UUID REFERENCES card_trios(id), -- null si D1, trio D(n-1)
sinon
  difficulty       FLOAT DEFAULT 0.5,

  UNIQUE (emettrice_id, cable_id, receptrice_id)
);

CREATE INDEX idx_trios_emettrice  ON card_trios(emettrice_id);
```

```
CREATE INDEX idx_trios_cable      ON card_trios(cable_id);
CREATE INDEX idx_trios_receptrice ON card_trios(receptrice_id);
CREATE INDEX idx_trios_distance   ON card_trios(distance_level);
```

3.4 Autres tables clés

```
-- Table des profils utilisateurs
CREATE TABLE user_profiles (
  id                UUID PRIMARY KEY REFERENCES auth.users(id),
  username          TEXT UNIQUE NOT NULL,
  avatar_url        TEXT,
  total_score       INT     DEFAULT 0,
  current_level     INT     DEFAULT 1,
  lives             INT     DEFAULT 5,
  lives_last_refill TIMESTAMPTZ DEFAULT now(),
  created_at        TIMESTAMPTZ DEFAULT now()
);

-- Table des codes d'activation physiques
CREATE TABLE activation_codes (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code_value   TEXT UNIQUE NOT NULL,
  is_activated BOOLEAN DEFAULT FALSE,
  device_id    TEXT,
  user_id      UUID REFERENCES user_profiles(id),
  activated_at TIMESTAMPTZ
);

-- Table des sessions de jeu
CREATE TABLE game_sessions (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID REFERENCES user_profiles(id),
  level_number     INT  NOT NULL,
  score            INT  DEFAULT 0,
  correct_answers  INT  DEFAULT 0,
  wrong_answers    INT  DEFAULT 0,
  bonus_earned     INT  DEFAULT 0,
  malus_received   INT  DEFAULT 0,
  duration_seconds INT,
  completed        BOOLEAN DEFAULT FALSE,
  started_at       TIMESTAMPTZ DEFAULT now(),
  ended_at         TIMESTAMPTZ
);
```

4. ALGORITHME DE GÉNÉRATION DES CARTES — PAS À PAS

Cette section détaille comment créer les cartes et les insérer en base, en partant du dessin jusqu'à l'URL. Fil rouge : la chaîne du Lion jusqu'à distance 3.

4.1 Étape 0 — Créer et uploader l'image Émettrice racine

1. Le chef artistique dessine le lion de base → fichier : lion_base.webp
2. Upload dans Supabase Storage : bucket "trialgo-cards", chemin "emettrices/savane/lion_base.webp"
3. Insérer l'entrée en base

```
-- INSERT Émettrice racine D1
INSERT INTO cards (
  card_type, distance_level,
  image_path,
  image_width, image_height, image_format,
  theme_tags,
  parent_emettrice_id, parent_cable_id, root_emettrice_id
) VALUES (
  'emettrice', 1,
  'emettrices/savane/lion_base.webp',
  512, 512, 'webp',
  ARRAY['animal', 'lion', 'savane', 'felin'],
  NULL, NULL, NULL   -- racine : aucun parent
) RETURNING id;
-- → suppose que l'id retourné est : <id_lion_base>
```

Résultat — Émettrice racine

id : <id_lion_base>
card_type : "emettrice"
distance_level : 1
image_path : "emettrices/savane/lion_base.webp"
imageUrl (client): https://<ref>.supabase.co/.../emettrices/savane/lion_base.webp
parent_* : NULL (c'est une racine)

4.2 Étape 1 — Créer le Câble-image et la Réceptrice D1

Le Câble est une illustration qui représente la transformation "miroir horizontal". C'est une image dessinée avec deux flèches symétriques. La Réceptrice est le lion dessiné en miroir.

4. Dessiner le câble "miroir H" → fichier : miroir_h.webp
5. Upload : "cables/geometrique/miroir_h.webp"

6. Dessiner la réceptrice "lion miroir" → fichier : lion_miroir_h.webp
7. Upload : "receptrices/savane/d1/lion_miroir_h.webp"
8. Insérer les deux cartes et le trio D1 en base

```
-- INSERT Câble-image "Miroir H"
INSERT INTO cards (
  card_type, distance_level,
  image_path,
  image_width, image_height, image_format,
  cable_category,
  theme_tags
) VALUES (
  'cable', 1,
  'cables/geometrique/miroir_h.webp',
  512, 512, 'webp',
  'geometrique',
  ARRAY['miroir', 'symetrie', 'geometrie']
) RETURNING id;
-- → <id_cable_miroir_h>

-- INSERT Réceptrice D1 : lion miroir
INSERT INTO cards (
  card_type, distance_level,
  image_path,
  image_width, image_height, image_format,
  theme_tags,
  parent_emettrice_id,
  parent_cable_id,
  root_emettrice_id
) VALUES (
  'receptrice', 1,
  'receptrices/savane/d1/lion_miroir_h.webp',
  512, 512, 'webp',
  ARRAY['animal', 'lion', 'savane', 'miroir'],
  <id_lion_base>,       -- parent_emettrice_id = Émettrice racine
  <id_cable_miroir_h>,  -- parent_cable_id = le Câble-image utilisé
  NULL                  -- root_emettrice_id null pour D1
) RETURNING id;
-- → <id_lion_miroir_h>

-- INSERT Trio D1
INSERT INTO card_trios (emettrice_id, cable_id, receptrice_id, distance_level)
VALUES (<id_lion_base>, <id_cable_miroir_h>, <id_lion_miroir_h>, 1);
```

Résultat — Trio D1 complet (3 images)

Émettrice (image 1) : lion_base.webp
URL : .../emettrices/savane/lion_base.webp

Câble (image 2) : miroir_h.webp ← c'est une IMAGE de deux flèches symétriques
URL : .../cables/geometrique/miroir_h.webp

Réceptrice(image 3) : lion_miroir_h.webp
URL : .../receptrices/savane/d1/lion_miroir_h.webp

Relation : image 1 ⊕image 2 = image 3
Points : G(3) = 3 - 1 = 2 pts

4.3 Étape 2 — Extension vers Distance 2

La Réceptrice D1 (lion miroir) joue maintenant le rôle d'Émettrice D2. Un nouveau Câble-image encode la seconde transformation. La nouvelle Réceptrice D2 est dessinée.

9. Dessiner le câble "teinte rouge" → fichier : teinte_rouge.webp
10. Upload : "cables/couleur/teinte_rouge.webp"
11. Dessiner la réceptrice D2 "lion miroir rouge" → fichier : lion_miroir_h_teinte_rouge.webp
12. Upload : "receptrices/savane/d2/lion_miroir_h_teinte_rouge.webp"
13. Insérer et enregistrer le trio D2 lié au trio D1

```
-- INSERT Câble-image "Teinte rouge"
INSERT INTO cards (card_type, distance_level, image_path, cable_category,
theme_tags)
VALUES ('cable', 2, 'cables/couleur/teinte_rouge.webp', 'couleur',
        ARRAY['rouge', 'teinte', 'couleur']) RETURNING id;
-- → <id_cable_teinte_rouge>

-- INSERT Réceptrice D2 : lion miroir rouge
-- IMPORTANT :
--   parent_emettrice_id → la Réceptrice D1 (qui joue Émettrice D2)
--   parent_cable_id     → le Câble-image D2
--   root_emettrice_id   → l'Émettrice racine originale (lion_base)
INSERT INTO cards (
  card_type, distance_level, image_path,
  image_width, image_height, image_format,
  theme_tags,
  parent_emettrice_id,
  parent_cable_id,
  root_emettrice_id
) VALUES (
  'receptrice', 2,
  'receptrices/savane/d2/lion_miroir_h_teinte_rouge.webp',
  512, 512, 'webp',
  ARRAY['animal', 'lion', 'savane', 'miroir', 'rouge'],
  <id_lion_miroir_h>,       -- Réceptrice D1 joue le rôle d'Émettrice ici
  <id_cable_teinte_rouge>,   -- Câble-image D2
  <id_lion_base>             -- Émettrice racine de la chaîne
) RETURNING id;
-- → <id_lion_miroir_rouge>

-- INSERT Trio D2
INSERT INTO card_trios (emettrice_id, cable_id, receptrice_id, distance_level,
parent_trio_id)
```

```
VALUES (<id_lion_miroir_h>, <id_cable_teinte_rouge>, <id_lion_miroir_rouge>, 2,
<id_trio_D1>);
```

Résultat — Quintette D1+D2 (5 images)

Image 1 : lion_base.webp (Émettrice D1)
Image 2 : miroir_h.webp (Câble D1 — flèches symétriques)
Image 3 : lion_miroir_h.webp (Réceptrice D1 = Émettrice D2)
Image 4 : teinte_rouge.webp (Câble D2 — palette rouge)
Image 5 : lion_miroir_h_teinte_rouge.webp (Réceptrice D2)

Chaque image a son propre chemin Storage, son propre UUID en base.
Points quintette : G(5) = 4 pts

4.4 Étape 3 — Extension vers Distance 3 (Septette)

14. Dessiner le câble "fragmentation 3" → fichier : fragmentation_3.webp
15. Upload : "cables/complexe/fragmentation_3.webp"
16. Dessiner la réceptrice D3 → fichier : lion_miroir_h_teinte_rouge_fragmente.webp
17. Upload : "receptrices/savane/d3/lion_miroir_h_teinte_rouge_fragmente.webp"
18. Insérer et créer le trio D3 lié au trio D2

Septette complet (7 images) — chaîne Lion

Image 1 : emettrices/savane/lion_base.webp
Image 2 : cables/geometrique/miroir_h.webp
Image 3 : receptrices/savane/d1/lion_miroir_h.webp
Image 4 : cables/couleur/teinte_rouge.webp
Image 5 : receptrices/savane/d2/lion_miroir_h_teinte_rouge.webp
Image 6 : cables/complexe/fragmentation_3.webp
Image 7 : receptrices/savane/d3/lion_miroir_h_teinte_rouge_fragmente.webp

7 fichiers uploadés · 7 enregistrements en base · 3 trios enregistrés · 1 chaîne complète
Points septette : G(7) = 6 pts

4.5 Fonction de validation de cohérence

La validation d'un trio se fait toujours par requête dans la table card_trios, qui est la source de vérité de toutes les relations valides entre images.

```
-- POSTGRESQL FUNCTION — Validation d'un trio d'images
```

```
CREATE OR REPLACE FUNCTION is_coherent(
  p_emettrice UUID,
  p_cable     UUID,
  p_receptrice UUID
) RETURNS BOOLEAN AS $$
BEGIN
  RETURN EXISTS (
    SELECT 1 FROM card_trios
    WHERE emettrice_id  = p_emettrice
      AND cable_id      = p_cable
      AND receptrice_id = p_receptrice
  );
END;
$$ LANGUAGE plpgsql STABLE;

-- Test sur notre trio D1
SELECT is_coherent(<id_lion_base>, <id_cable_miroir_h>, <id_lion_miroir_h>);
-- → TRUE

-- Test avec le mauvais câble
SELECT is_coherent(<id_lion_base>, <id_cable_teinte_rouge>, <id_lion_miroir_h>);
-- → FALSE (teinte_rouge ne produit pas lion_miroir_h depuis lion_base)
```

```
// DART — ValidateTripletUseCase
(lib/domain/usecases/validate_triplet_usecase.dart)

class ValidateTripletUseCase {
  final CardTrioRepository repository;
  ValidateTripletUseCase(this.repository);

  // Valide que trois images forment bien un trio cohérent
  // Peu importe la "config" (A/B/C) — on passe toujours les 3 IDs dans le bon
ordre
  Future<bool> call({
    required String emettriceId,   // l'image Émettrice (ou celle jouant ce rôle)
    required String cableId,        // l'image Câble
    required String receptriceId,   // l'image Réceptrice (résultat)
  }) {
    return repository.isCoherent(
      emettriceId:  emettriceId,
      cableId:      cableId,
      receptriceId: receptriceId,
    );
  }
}
```

5. AFFICHAGE DES IMAGES DANS FLUTTER

Toutes les images étant des URL Supabase Storage, le chargement se fait via un widget Flutter standard avec mise en cache agressive. Le chemin relatif stocké en base est converti en URL complète par StorageConstants.

5.1 Widget CardImageWidget

```
// lib/presentation/widgets/card_image_widget.dart

class CardImageWidget extends StatelessWidget {
  final CardEntity card;
  final double size;
  final bool isSelected;
  final bool isMasked;    // true = afficher "???" à la place
  final VoidCallback? onTap;

  const CardImageWidget({
    required this.card,
    this.size = 120,
    this.isSelected = false,
    this.isMasked = false,
    this.onTap,
    super.key,
  });

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 200),
        width: size,
        height: size,
        decoration: BoxDecoration(
          border: Border.all(
            color: isSelected ? Colors.amber : Colors.transparent,
            width: 3,
          ),
          borderRadius: BorderRadius.circular(12),
          boxShadow: isSelected ? [BoxShadow(color: Colors.amber.withOpacity(0.5),
            blurRadius: 8, spreadRadius: 2)] : [],
        ),
        child: ClipRRect(
          borderRadius: BorderRadius.circular(10),
          child: isMasked
            ? _buildMaskedCard()
            : CachedNetworkImage(
                // card.imageUrl reconstruit l'URL depuis image_path
                imageUrl: card.imageUrl,
                fit: BoxFit.cover,
                placeholder: (_, __) =>
                  const Center(child: CircularProgressIndicator()),
```

```
                errorWidget: (_, __, ___) =>
                  const Center(child: Icon(Icons.broken_image)),
              ),
          ),
        ),
      );
    }

    Widget _buildMaskedCard() => Container(
      color: Colors.grey[800],
      child: const Center(
        child: Text('?', style: TextStyle(fontSize: 48, color: Colors.white)),
      ),
    );
  }
```

5.2 Gestion du cache et préchargement

Pour éviter tout lag à l'affichage, les images sont préchargées dès que la question est générée, avant même que l'écran ne s'affiche.

```
// Préchargement dans GenerateQuestionUseCase
Future<void> _precacheImages(
  BuildContext context,
  GameQuestion question,
) async {
  final allCards = [...question.visibleCards, ...question.choices];
  await Future.wait(
    allCards.map((card) => precacheImage(
      CachedNetworkImageProvider(card.imageUrl), context
    )),
  );
}
```

6. MÉCANIQUE DE JEU — MODE SOLO EN LIGNE

6.1 Vue d'ensemble de l'écran de jeu

L'écran de jeu affiche deux images de cartes en haut. Une troisième position affiche "???" — c'est la carte masquée. En bas, une ScrollView horizontale présente 10 images parmi lesquelles choisir.

Interface principale — toutes les positions sont des images

```
┌──────────────────────────────────────────────────────────────┐
│ [NIV. 7]  ♥♥♥♡♡  SCORE: 1,240  ⏱ 00:38                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ [IMAGE E]    │  │ [IMAGE C]    │  │    ???       │       │
│  │ lion_base    │  │ miroir_h     │  │ À trouver    │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│  Émettrice         Câble (image)     Réceptrice              │
│                                                              │
│  "Quelle image complète ce trio ?"                           │
├──────────────────────────────────────────────────────────────┤
│ ← faire défiler les 10 images →                              │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐      │
│  │[img1]│ │[img2]│ │[img3]│ │[img4]│ │[img5]│ │[img6]│      │
│  └──────┘ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘      │
└──────────────────────────────────────────────────────────────┘
```

Toutes les "cases" de l'écran (haut et bas) sont des CardImageWidget.
La ScrollView affiche des images de cartes de même type que la carte masquée.

6.2 Les trois configurations de question

Config.	Image 1 visible	Image 2 visible	Image à trouver	Niveaux	Difficulté
A	Émettrice	Câble (image)	Réceptrice	1 – 5	Faible
B	Émettrice	Réceptrice	Câble (image)	4 – 10	Moyenne
C	Câble (image)	Réceptrice	Émettrice	8 – 20+	Élevée

Exemple config. B — trouver le Câble-image

Image visible 1 : lion_base.webp (Émettrice)
Image visible 2 : lion_miroir_h.webp (Réceptrice)

Question : "Quelle image-câble relie ces deux images ?"
Réponse : miroir_h.webp (image des deux flèches symétriques)

→ Les 9 distracteurs sont d'autres images de câbles (geometrique, couleur, etc.)
Le joueur doit reconnaître visuellement quelle transformation relie E à R.

6.3 Génération des distracteurs — par type d'image

Les distracteurs de la ScrollView sont toujours du même type que la carte masquée. Si la carte masquée est un Câble, tous les distracteurs sont des images de câbles. Si c'est une Émettrice, ce sont d'autres images d'émettrices.

Carte masquée	Type des distracteurs	Critère de proximité
Réceptrice	9 autres images de réceptrices	Même distance_level en priorité, puis même theme_tags
Câble	9 autres images de câbles	Même cable_category en priorité, puis câbles d'autres catégories
Émettrice	9 autres images d'émettrices	Même theme_tags partiels en priorité, puis thèmes différents

```dart
// DART — DistractorService (lib/data/services/distractor_service.dart)
Future<List<CardEntity>> generateDistractors({
  required CardEntity correctCard,
  required int level,
  int count = 9,
}) async {
  // On filtre toujours sur le même card_type que la carte correcte
  var query = supabase
    .from('cards')
    .select()
    .eq('card_type', correctCard.cardType.name)
    .eq('is_active', true)
    .neq('id', correctCard.id);

  // Pour les câbles : filtrer par catégorie proche en premier
  if (correctCard.isCable && correctCard.cableCategory != null) {
    final sameCategory = await query
      .eq('cable_category', correctCard.cableCategory!)
      .limit(count ~/ 2)
      .withConverter((d) => d.map(CardModel.fromJson).toList());
    // Compléter avec d'autres catégories si besoin
    final otherCategory = await query
      .neq('cable_category', correctCard.cableCategory!)
      .limit(count - sameCategory.length)
      .withConverter((d) => d.map(CardModel.fromJson).toList());
    return [...sameCategory, ...otherCategory]..shuffle();
  }

  // Pour les émettrices et réceptrices : filtrer par distance et theme
  return query
```

```
    .eq('distance_level', correctCard.distanceLevel)
    .limit(count)
    .withConverter((d) => d.map(CardModel.fromJson).toList());
}
```

7. SYSTÈME DE NIVEAUX — PROGRESSION DÉTAILLÉE

7.1 Tableau de progression

Niveaux	Distance	Config.	Questions	Seuil	Vies/question	Temps/tour	Pts base
1 – 3	D1	A	8	6/8	3	30s	10
4 – 6	D1	A+B	10	7/10	3	35s	15
7 – 10	D1+D2	A+B	10	7/10	2	40s	20
11 – 14	D2	B	10	7/10	2	45s	25
15 – 18	D2+D3	B+C	12	8/12	2	50s	35
19 – 22	D3	C	12	9/12	1	55s	50
23+	D3	A+B+C	15	11/15	1	45s	75

7.2 Score de niveau

Formule de score

Score = Points_base × Multiplicateur_distance × Bonus_temps

Multiplicateur : D1 × 1.0 | D2 × 1.5 | D3 × 2.0

Bonus temps :

Réponse dans les 25% du temps → × 1.5 (Turbo)
Réponse entre 25% et 50% → × 1.25 (Rapide)
Réponse entre 50% et 75% → × 1.0 (Normal)
Réponse après 75% → × 0.75 (Lent)

8. SYSTÈME BONUS, MALUS & DÉLIMITEUR DE TEMPS

8.1 Délimiteur de temps de tour

Comportement du chronomètre de tour

DÉMARRAGE : Dès que les images de la question sont chargées (pas avant).
AFFICHAGE : Barre circulaire animée. Couleurs : vert → orange → rouge.
> 60% restant → Vert
30% – 60% → Orange
< 30% → Rouge + vibration légère
EXPIRATION :
→ Question considérée incorrecte automatiquement
→ MALUS_TIMEOUT appliqué
→ La bonne image se met en surbrillance 1.5s
→ Passage à la question suivante après 2s

8.2 Délimiteur de temps de session

Niveaux	Durée max session	Avertissement	À l'expiration
1 – 5	10 minutes	À 2 min restantes	Session fermée, score sauvegardé
6 – 10	15 minutes	À 3 min restantes	Session fermée, score sauvegardé
11 – 15	20 minutes	À 4 min restantes	Session fermée, score sauvegardé
16+	25 minutes	À 5 min restantes	Session fermée, score sauvegardé

8.3 Catalogue des BONUS

Bonus	Code	Déclencheur	Effet
Turbo	BONUS_TURBO	Bonne réponse dans les 25% du temps	+ 50% pts sur la question
Série	BONUS_STREAK	3 bonnes réponses consécutives	+ 20 pts par réponse suivante
Grande Série	BONUS_MEGA_STREAK	7 bonnes réponses d'affilée	+ 10s chrono sur 3 questions
Sans Faute	BONUS_PERFECT	Niveau entier sans erreur	Score du niveau × 2
Expert	BONUS_EXPERT	Trouver l'Émettrice (config C)	+ 30 pts + badge
Vitesse	BONUS_SPEED_RUN	5 questions < 15s chacune	+ 1 vie
Économe	BONUS_LIFESAVER	Niveau sans perdre de vie	+ 100 pts + 1 vie

8.4 Catalogue des MALUS

Malus	Code	Déclencheur	Effet
Temps écoulé	MALUS_TIMEOUT	Chrono tour = 0	0 pt + -1 vie + -5 pts session
Mauvaise réponse	MALUS_WRONG	Image incorrecte sélectionnée	-1 vie sur la question
Série brisée	MALUS_STREAK_BREAK	Erreur après streak ≥ 3	Streak réinitialisé
Double erreur	MALUS_INCOHERENT	2 erreurs sur la même question	-10 pts pénalité
Niveau échoué	MALUS_LEVEL_FAIL	< seuil de bonnes réponses	0 pts du niveau, replay
Session expirée	MALUS_SESSION_TIMEOUT	Durée session > maximum	Score gelé, session fermée

8.5 Système de vies — recharge progressive

Paramètre	Valeur
Stock initial	5 vies
Maximum	5 vies (jamais dépassé)
Recharge automatique	1 vie toutes les 30 minutes (pg_cron Supabase)
Coût mauvaise réponse	-1 vie
Coût timeout de tour	-1 vie
Vies = 0	Session impossible jusqu'à recharge
Bonus Économe	+1 vie si niveau sans perte
Bonus Vitesse	+1 vie si 5 questions rapides

9. ARCHITECTURE FLUTTER — CLEAN ARCHITECTURE + RIVERPOD

9.1 Structure de dossiers

```
lib/
├── core/
│   ├── constants/
│   │   ├── storage_constants.dart    # URL base + helpers chemins images
│   │   └── game_constants.dart        # Seuils, timeouts, limites
│   ├── error/
│   │   └── failures.dart              # CardFailure, AuthFailure, etc.
│   └── network/
│       └── supabase_client.dart        # Singleton Supabase
│
├── domain/
│   ├── entities/
│   │   ├── card_entity.dart            # cardType, imagePath, imageUrl getter...
│   │   ├── card_trio_entity.dart        # emettrice + cable + receptrice (3
CardEntity)
│   │   ├── game_question_entity.dart    # visibles, masquée, 10 choix, config
│   │   └── user_entity.dart
│   ├── repositories/
│   │   ├── card_repository.dart         # interface
│   │   ├── card_trio_repository.dart    # interface
│   │   └── game_session_repository.dart # interface
│   └── usecases/
│       ├── generate_question_usecase.dart
│       ├── validate_triplet_usecase.dart
│       └── activate_code_usecase.dart
│
├── data/
│   ├── models/
│   │   ├── card_model.dart              # fromJson : mappe image_path → imageUrl
│   │   └── card_trio_model.dart
│   ├── datasources/
│   │   ├── supabase_card_datasource.dart
│   │   └── supabase_auth_datasource.dart
│   ├── repositories/
│   │   └── card_repository_impl.dart    # implémentation concrète
│   └── services/
│       └── distractor_service.dart      # génère les 9 images distractrices
│
└── presentation/
    ├── providers/
    │   ├── game_session_provider.dart   # StateNotifier session
    │   ├── question_timer_provider.dart # chronomètre de tour
    │   ├── session_timer_provider.dart  # chronomètre de session
    │   └── lives_provider.dart          # StreamProvider vies
    ├── pages/
    │   ├── game_page.dart
    │   ├── auth_page.dart
    │   └── activation_page.dart
    └── widgets/
        ├── card_image_widget.dart        # affiche une image de carte
```

```
        ├── card_scroll_view.dart         # ScrollView des 10 images
        ├── timer_widget.dart
        └── lives_widget.dart
```

9.2 Providers Riverpod

Provider	Type	Rôle
authProvider	StateNotifierProvider	État d'authentification
activationProvider	StateNotifierProvider	Flux activation code
gameSessionProvider	StateNotifierProvider	Session en cours (score, vies, streak)
currentQuestionProvider	FutureProvider.family	Génère la question + 10 images
questionTimerProvider	StateNotifierProvider.autoDispose	Chronomètre de tour
sessionTimerProvider	StateNotifierProvider	Chronomètre global de session
bonusMalusProvider	StateNotifierProvider.autoDispose	Calcul bonus/malus temps réel
livesProvider	StreamProvider	Vies (Supabase Realtime)
leaderboardProvider	FutureProvider	Classement (cache 5 min)
userProfileProvider	StreamProvider	Profil utilisateur temps réel

10. AUTHENTIFICATION & ACTIVATION DU CODE DE JEU

10.1 Flux d'authentification

19. Écran d'accueil : "Créer un compte" ou "Se connecter"
20. Option 1 : Email + mot de passe (Supabase Auth)
21. Option 2 : "Continuer avec Google" (OAuth Google)
22. Vérification e-mail (inscription classique)
23. Connexion réussie → vérification code activé
24. Code non activé → écran de saisie du code obligatoire
25. Code activé → menu principal

10.2 Règle Device ID

Sécurité : 1 code d'activation = 1 appareil unique

Scénarios :
A) Code valide non activé → Activation + enregistrement Device ID
B) Code activé + même Device ID → Reconnexion autorisée (même téléphone)
C) Code activé + Device ID différent → REFUS "Code déjà utilisé sur un autre appareil"
D) Code inexistant → REFUS "Code invalide"

Validation via Supabase Edge Function (server-side) — non contournable côté client.

10.3 Edge Function — validation et activation

```
// supabase/functions/activate-code/index.ts
Deno.serve(async (req) => {
  const { code_value, user_id, device_id } = await req.json();

  const { data: code } = await supabase
    .from('activation_codes').select('*')
    .eq('code_value', code_value).single();

  if (!code) return error(404, "Code invalide");

  if (code.is_activated) {
    if (code.device_id === device_id) return ok({ message: "Reconnexion OK" });
    return error(403, "Code déjà utilisé sur un autre appareil");
  }

  await supabase.from('activation_codes').update({
    is_activated: true, device_id, user_id,
```

```
    activated_at: new Date().toISOString(),
  }).eq('code_value', code_value);

  return ok({ message: "Activé avec succès" });
});
```

11. RÉCAPITULATIF COMPLET — TOUS LES CAS DE FIGURE

Cas de figure	Mécanique déclenchée	Résultat
Upload d'une nouvelle carte (artiste)	Chef artistique → Supabase Storage → INSERT cards	Carte disponible dans le jeu après activation is_active=true
Chargement image Câble dans ScrollView	card.imageUrl depuis image_path + StorageConstants	CachedNetworkImage affiche l'illustration du câble
Câble non reconnu visuellement	Joueur sélectionne mauvaise image câble	isCoherent() = false → malus
Inscription email	Supabase Auth + vérification email	Compte créé → écran activation code
Connexion Google	OAuth Google via Supabase	Connexion directe → vérification code
Code valide non activé	Edge Function → Device ID enregistré	Accès au jeu déverrouillé
Code déjà activé même appareil	Edge Function → Device ID identique	Reconnexion autorisée
Code déjà activé autre appareil	Edge Function → Device ID différent	Accès bloqué — message d'erreur
Config A : trouver Réceptrice	GenerateQuestion → 10 images de réceptrices	Question facile, niveaux 1-5
Config B : trouver Câble-image	GenerateQuestion → 10 images de câbles	Question intermédiaire, niveaux 4-10
Config C : trouver Émettrice	GenerateQuestion → 10 images d'émettrices	Question difficile, niveaux 8+
Bonne réponse < 25% du temps	BONUS_TURBO	Score × 1.5
3 bonnes consécutives	BONUS_STREAK	+20 pts par réponse suivante
7 bonnes consécutives	BONUS_MEGA_STREAK	+10s chrono sur 3 questions
Niveau sans faute	BONUS_PERFECT	Score niveau × 2
Config C réussie	BONUS_EXPERT	+30 pts + badge
Image incorrecte choisie	MALUS_WRONG → -1 vie	0 pt, bonne image révélée 1.5s
Chrono de tour expiré	MALUS_TIMEOUT → -1 vie, -5 pts	Bonne image révélée, question suivante
Erreur après streak ≥ 3	MALUS_STREAK_BREAK	Streak remis à 0
2 erreurs même question	MALUS_INCOHERENT	-10 pts pénalité
Vies = 0	Session terminée	Score sauvegardé, recharge 30 min/vie
Niveau échoué	Replay obligatoire	0 pts, nouvelles images aléatoires
Session > durée max	MALUS_SESSION_TIMEOUT	Score actuel sauvegardé
30 min écoulées, vies < 5	pg_cron Supabase	+1 vie automatiquement

Ce document constitue la référence technique et fonctionnelle v3.0 du projet TRIALGO. ***Correction majeure : la carte Câble est une image à part entière. Toutes les cartes possèdent un chemin image_path dans Supabase Storage et une URL publique. La table algorithms abstraite est supprimée.***

12. WORKFLOW GLOBAL — DE LA CONNEXION AU JEU

Cette section présente le parcours complet d'un utilisateur depuis le premier lancement de l'application jusqu'à une partie en cours. Chaque étape du chemin nominal est documentée, ainsi que toutes les situations alternatives (erreurs, cas limites, retours en arrière).

12.1 Vue d'ensemble du workflow en 5 phases

Les 5 phases du parcours utilisateur

PHASE 1 — Démarrage de l'application
Splash screen · Vérification session active

PHASE 2 — Authentification
Inscription email/mdp OU connexion Google
Vérification email (si inscription)

PHASE 3 — Activation du code de jeu physique
Saisie du code · Vérification Device ID · Lien compte-appareil

PHASE 4 — Menu principal
Vérification profil · Chargement état joueur (niveau, vies, score)

PHASE 5 — Session de jeu
Génération question · Tour de jeu · Bonus/Malus · Fin de session

12.2 PHASE 1 — Démarrage de l'application

À chaque lancement, l'application vérifie silencieusement si une session Supabase est encore active en mémoire. Cette vérification détermine l'écran d'atterrissage.

Chemin nominal — session active trouvée

[Splash Screen]
↓ Supabase.auth.currentSession != null ?
↓ OUI
[Vérification code activé pour ce compte]
↓ activation_codes WHERE user_id = uid AND is_activated = true
↓ OUI
[Chargement profil utilisateur]
↓ SELECT * FROM user_profiles WHERE id = uid
[Menu principal]

Situations alternatives — Phase 1

ALT 1.A — Aucune session active
→ currentSession = null
→ Redirection vers Écran d'authentification (Phase 2)

ALT 1.B — Session active MAIS code non activé
→ Aucun enregistrement dans activation_codes pour ce user_id
→ Redirection directe vers Écran de saisie du code (Phase 3)
→ L'utilisateur ne peut pas atteindre le menu principal sans code valide

ALT 1.C — Session active, code activé, MAIS profil introuvable
→ user_profiles ne contient pas l'uid (corruption ou première connexion incomplète)
→ Création automatique du profil par défaut (INSERT user_profiles)
→ Redirection vers Menu principal

ALT 1.D — Pas de connexion réseau au démarrage
→ Supabase client retourne une erreur réseau
→ Affichage : "Pas de connexion internet. Veuillez vérifier votre réseau."
→ Bouton "Réessayer" qui relance la vérification de session

12.3 PHASE 2 — Authentification

L'utilisateur choisit entre inscription et connexion. Deux chemins coexistent : email/mot de passe (natif Supabase) et Google OAuth. Les situations alternatives couvrent les erreurs fréquentes.

Chemin nominal — Inscription email

[Écran Auth] → Bouton "Créer un compte"
↓
[Formulaire] saisie email + mot de passe + confirmation
↓ Validation locale (email format, mdp ≥ 8 car., correspondance)
↓ POST /auth/v1/signup {email, password}
↓ Supabase envoie un email de vérification
[Écran "Vérifiez votre email"]
↓ L'utilisateur clique le lien dans l'email
↓ Deep Link capturé par l'app Flutter
↓ Supabase confirme → session créée automatiquement
↓ INSERT user_profiles (id, username par défaut)
[Écran saisie du code de jeu] (Phase 3)

Chemin nominal — Connexion Google

[Écran Auth] → Bouton "Continuer avec Google"
↓ Flutter lance supabase.auth.signInWithOAuth(OAuthProvider.google)
↓ Navigateur système s'ouvre → page consent Google
↓ L'utilisateur accepte
↓ Redirect URI capturé → Deep Link → retour dans l'app
↓ Session Supabase créée
↓ Vérification si user_profiles existe pour cet uid
↓ Si NON → INSERT user_profiles automatique
↓ Vérification code activé → si absent → Phase 3
→ si présent → Phase 4

Situations alternatives — Phase 2

ALT 2.A — Email déjà utilisé (inscription)
→ Supabase retourne : "User already registered"
→ Affichage : "Cet email est déjà associé à un compte. Connectez-vous."
→ Lien vers l'écran de connexion

ALT 2.B — Mot de passe trop faible
→ Validation locale avant appel : longueur < 8 car.
→ Affichage inline sous le champ : "Minimum 8 caractères requis"
→ Aucun appel réseau effectué

ALT 2.C — Email ou mot de passe incorrect (connexion)
→ Supabase retourne : "Invalid login credentials"
→ Affichage : "Email ou mot de passe incorrect"
→ Lien "Mot de passe oublié ?" → envoi email reset

ALT 2.D — Email non vérifié (connexion après inscription sans clic lien)
→ Supabase retourne : "Email not confirmed"
→ Affichage : "Email non confirmé. Un nouveau lien a été envoyé."
→ Appel automatique POST /auth/v1/resend pour renvoyer l'email

ALT 2.E — Connexion Google annulée
→ L'utilisateur ferme le navigateur sans accepter
→ Retour sur l'écran d'authentification sans erreur affichée

ALT 2.F — Connexion Google : compte Google déjà lié à un autre email Supabase
→ Supabase retourne une erreur de conflit d'identité
→ Affichage : "Ce compte Google est déjà lié à un autre profil."

12.4 PHASE 3 — Activation du code de jeu physique

Cette phase est obligatoire après toute première connexion. Elle lie définitivement un code physique imprimé dans la boîte du jeu à un compte et à un appareil.

Chemin nominal — Activation réussie

[Écran saisie du code]
↓ L'utilisateur saisit le code (ex: "TRLG-4X9K-2M7P")
↓ Validation locale : format correct (16 car. alphanum. tirets)
↓ Récupération Device ID (device_info_plus)
↓ POST /functions/v1/activate-code {code_value, user_id, device_id}
↓ Edge Function : vérification DB → code valide + non activé
↓ UPDATE activation_codes SET is_activated=true, device_id, user_id
↓ Réponse 200 OK
[Animation de succès "Code activé !"]
↓
[Menu principal] (Phase 4)

Situations alternatives — Phase 3

ALT 3.A — Code invalide (n'existe pas en base)
→ Edge Function retourne 404
→ Affichage : "Code introuvable. Vérifiez le code inscrit dans votre boîte."
→ Champ de saisie vidé et mis en rouge

ALT 3.B — Code déjà utilisé sur un autre appareil
→ Edge Function retourne 403
→ Affichage : "Ce code a déjà été activé sur un autre appareil."
→ Message d'aide : "Contactez le support si vous avez changé de téléphone."

ALT 3.C — Même code, même appareil (reconnexion après réinstallation)
→ Edge Function détecte device_id identique → retourne 200 "Reconnexion OK"
→ L'application traite ce retour comme une activation réussie
→ Redirection vers Menu principal

ALT 3.D — Format de code incorrect (validation locale)
→ Regex de validation avant tout appel réseau
→ Affichage : "Format incorrect. Le code contient 16 caractères."
→ Aucun appel réseau effectué

ALT 3.E — Erreur réseau pendant l'activation
→ Timeout ou absence de connexion
→ Affichage : "Erreur réseau. Vérifiez votre connexion et réessayez."
→ Bouton "Réessayer" disponible, code conservé dans le champ

12.5 PHASE 4 — Menu principal

Le menu principal est le hub de navigation. À son affichage, l'application charge l'état complet du joueur depuis Supabase en temps réel.

Chemin nominal — Chargement du menu

[Menu principal affiché]
↓ StreamProvider livesProvider → SELECT lives FROM user_profiles
↓ StreamProvider userProfileProvider → SELECT * FROM user_profiles
↓ FutureProvider leaderboardProvider → SELECT ... FROM user_profiles ORDER BY total_score DESC
↓ Supabase Realtime activé sur user_profiles pour ce user_id
↓ Affichage : niveau actuel, vies disponibles, score total

Options disponibles :
→ [Jouer] Démarrage Phase 5
→ [Galerie] Cartes débloquées
→ [Classement] Leaderboard global
→ [Paramètres] Profil, son, notifications

Situations alternatives — Phase 4

ALT 4.A — Vies = 0 au moment d'appuyer sur "Jouer"
→ Vérification lives > 0 avant de lancer la session
→ Affichage modal : "Plus de vies ! Prochaine recharge dans X minutes."
→ Affichage du timer de recharge en temps réel
→ Blocage du bouton "Jouer" jusqu'à recharge

ALT 4.B — Perte de connexion en fond de menu
→ Supabase Realtime perd la connexion
→ Bannière subtile : "Connexion perdue - données possiblement non à jour"
→ Tentative de reconnexion automatique toutes les 30s

ALT 4.C — Déconnexion volontaire
→ Bouton dans Paramètres → supabase.auth.signOut()
→ Nettoyage de tous les providers Riverpod
→ Redirection vers Écran d'authentification (Phase 2)
→ La prochaine connexion repasse par Phase 3 si même device_id

12.6 PHASE 5 — Session de jeu

Une session de jeu est une séquence de questions pour un niveau donné. Elle a un début, une durée maximale, et se termine par une réussite, un échec, ou une expiration.

Chemin nominal — Tour de jeu réussi

[Démarrage session]
- ↓ INSERT game_sessions {user_id, level_number, started_at}
- ↓ GenerateQuestionUseCase : SELECT trio aléatoire + 9 distracteurs
- ↓ Préchargement des 10 images (CachedNetworkImage)
- ↓ start sessionTimerProvider (durée max selon niveau)

[Affichage question]
- ↓ start questionTimerProvider (chrono de tour)
- ↓ Le joueur fait défiler la ScrollView → identifie l'image
- ↓ Le joueur tape sur une image
- ↓ stop questionTimerProvider
- ↓ ValidateTripletUseCase → SELECT card_trios WHERE E+C+R
- ↓ Résultat : TRUE
- ↓ BonusService.evaluate() → calcul points + bonus éventuels
- ↓ UPDATE game_sessions SET score, correct_answers, bonus_earned
- ↓ Animation "Bonne réponse !" → Question suivante

[Fin de niveau — seuil atteint]
- ↓ correct_answers >= seuil_niveau
- ↓ UPDATE game_sessions SET completed=true, ended_at
- ↓ UPDATE user_profiles SET current_level = current_level + 1
- ↓ Écran récapitulatif : score, bonus, étoiles obtenus

Situations alternatives — Phase 5

ALT 5.A — Mauvaise réponse
- → ValidateTripletUseCase retourne FALSE
- → MalusService : lives - 1, streak = 0
- → Bonne image révélée 1.5s en surbrillance verte
- → UPDATE game_sessions SET wrong_answers++, malus_received++
- → Question suivante après 2s

ALT 5.B — Chrono de tour expiré
- → questionTimerProvider.isExpired = true
- → Traitement identique à mauvaise réponse + malus timeout
- → -1 vie, -5 pts session

ALT 5.C — Vies = 0 en cours de session
- → MalusService détecte lives <= 0 après déduction
- → Session forcée terminée immédiatement
- → UPDATE game_sessions SET completed=false, ended_at
- → UPDATE user_profiles SET lives=0

→ Écran "Plus de vies" avec timer de recharge

ALT 5.D — Seuil de niveau non atteint (échec)
→ Fin des questions : correct_answers < seuil
→ Écran "Niveau échoué" : score affiché, meilleur score conservé
→ UPDATE game_sessions SET completed=false
→ Le niveau n'est PAS incrémenté dans user_profiles
→ Bouton "Rejouer" → nouvelle session avec nouvelles questions aléatoires

ALT 5.E — Durée maximale de session expirée
→ sessionTimerProvider.isExpired = true
→ Session fermée même si des questions restent
→ Score actuel sauvegardé tel quel
→ UPDATE game_sessions SET ended_at
→ Affichage "Temps de session écoulé" avec le score final

ALT 5.F — Perte de connexion en cours de jeu
→ Supabase client ne peut plus écrire
→ Les réponses sont mises en file locale (queue Flutter)
→ Bannière non intrusive : "Mode hors-ligne - score synchronisé dès reconnexion"
→ À la reconnexion : flush de la queue vers Supabase
→ Si reconnexion impossible avant fin de session : score perdu, message d'excuse

12.7 Diagramme de flux résumé

Flux global — toutes phases

```
LANCEMENT APP
  ├─ Session active + code activé ──────────────────→ MENU PRINCIPAL
  ├─ Session active + code NON activé ──────────────→ SAISIE CODE
  └─ Aucune session ──────────────────────────────────→ AUTHENTIFICATION

AUTHENTIFICATION
  ├─ Inscription email ──→ Vérif. email ──→ SAISIE CODE
  ├─ Connexion email ────→ OK ──────────────→ SAISIE CODE (si 1ère fois)
  │                          └─→ MENU PRINCIPAL (si déjà activé)
  └─ Connexion Google ───→ OK ──────────────→ idem ci-dessus

SAISIE CODE
  ├─ Code valide + device libre ─→ Activation ────────→ MENU PRINCIPAL
  ├─ Code valide + même device ──→ Reconnexion OK ──→ MENU PRINCIPAL
  ├─ Code valide + autre device ─→ REFUS
  └─ Code invalide ──────────────→ Erreur + retry
```

```
MENU PRINCIPAL
  ├─ Vies > 0 ──────────────────────────────────────────→ SESSION DE JEU
  └─ Vies = 0 ──────────────────────────────────────────→ Attente recharge

SESSION DE JEU
  ├─ Bonne réponse ──→ score++ ──→ question suivante
  ├─ Mauvaise réponse → vie-- ───→ question suivante
  ├─ Chrono expiré ──→ vie-- ───→ question suivante
  ├─ Vies = 0 ────────────────────→ Session terminée
  ├─ Seuil atteint ───────────────→ Niveau réussi ──→ MENU PRINCIPAL
  ├─ Seuil non atteint ───────────→ Niveau échoué ──→ Replay
  └─ Temps session expiré ────────→ Score sauvegardé → MENU PRINCIPAL
```

13. CONTRAT D'API BACKEND — APPELS ET RÉPONSES

Cette section constitue le contrat technique entre le frontend Flutter et le backend Supabase. Chaque appel est documenté avec sa méthode, son endpoint, le corps de la requête, tous les codes de réponse possibles et la structure exacte des données retournées.

Conventions

Base URL Supabase : https://<PROJECT_REF>.supabase.co
Auth header : Authorization: Bearer <JWT_TOKEN> (requis sauf indication)
Content-Type : application/json
Toutes les dates : format ISO 8601 (ex: "2025-09-15T14:32:00Z")
Tous les IDs : UUID v4 (ex: "550e8400-e29b-41d4-a716-446655440000")

13.1 Authentification

API-01 — Inscription par email et mot de passe

Champ	Valeur
Méthode + Endpoint	POST /auth/v1/signup
Auth requis	NON — appel non authentifié
SDK Flutter	supabase.auth.signUp(email: email, password: password)

```
// REQUÊTE
{
  "email":    "joueur@example.com",
  "password": "MonMotDePasse123"
}

// RÉPONSE 200 — Succès, email de vérification envoyé
{
  "user": {
    "id":    "550e8400-e29b-41d4-a716-446655440000",
    "email": "joueur@example.com",
    "email_confirmed_at": null,    // null = email pas encore vérifié
    "created_at": "2025-09-15T14:32:00Z"
  },
  "session": null    // null = session pas encore créée (en attente vérification)
}

// RÉPONSE 400 — Email déjà utilisé
{
  "code":    "user_already_exists",
  "message": "User already registered"
}
```

```
// RÉPONSE 422 — Mot de passe trop faible (< 6 car. côté Supabase)
{
  "code":    "weak_password",
  "message": "Password should be at least 6 characters"
}
```

API-02 — Connexion par email et mot de passe

Champ	Valeur
Méthode + Endpoint	POST /auth/v1/token?grant_type=password
Auth requis	NON
SDK Flutter	supabase.auth.signInWithPassword(email: email, password: password)

```
// REQUÊTE
{
  "email":    "joueur@example.com",
  "password": "MonMotDePasse123"
}

// RÉPONSE 200 — Connexion réussie
{
  "access_token":  "eyJhbGciOiJIUzI1NiIsInR...",   // JWT valable 1h
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2g...",   // valable 7 jours
  "token_type":    "bearer",
  "expires_in":    3600,
  "user": {
    "id":                "550e8400-...",
    "email":             "joueur@example.com",
    "email_confirmed_at": "2025-09-15T14:00:00Z"
  }
}

// RÉPONSE 400 — Identifiants incorrects
{
  "code":    "invalid_credentials",
  "message": "Invalid login credentials"
}

// RÉPONSE 400 — Email non confirmé
{
  "code":    "email_not_confirmed",
  "message": "Email not confirmed"
}
```

API-03 — Connexion via Google OAuth

Champ	Valeur

Méthode	Redirection OAuth (pas un appel JSON direct)
SDK Flutter	supabase.auth.signInWithOAuth(OAuthProvider.google, redirectTo: appScheme)
Résultat	Deep Link capturé → session créée automatiquement par le SDK

```
// Après retour du Deep Link, le SDK expose la session :

// SUCCÈS — session disponible via supabase.auth.currentSession
{
  "access_token":  "eyJhbGciOiJIUzI1NiIsInR...",
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2g...",
  "user": {
    "id":           "660e9500-...",
    "email":        "joueur@gmail.com",
    "app_metadata": { "provider": "google" }
  }
}

// ÉCHEC — annulation ou erreur Google
// supabase.auth.currentSession reste null
// Aucune exception levée — vérifier currentSession après retour
```

API-04 — Renvoi de l'email de vérification

```
// POST /auth/v1/resend
// REQUÊTE
{
  "type":  "signup",
  "email": "joueur@example.com"
}

// RÉPONSE 200 — Email renvoyé
{}    // Corps vide — succès implicite

// RÉPONSE 429 — Trop de renvois (rate limit)
{
  "code":    "over_email_send_rate_limit",
  "message": "Email rate limit exceeded"
}
```

API-05 — Déconnexion

```
// POST /auth/v1/logout
// Auth requis : OUI (Bearer JWT)
// SDK Flutter : supabase.auth.signOut()

// RÉPONSE 204 — Déconnexion réussie (corps vide)
```

```
// RÉPONSE 401 — Token invalide ou expiré
{
  "code":    "invalid_jwt",
  "message": "JWT expired"
}
// → Dans ce cas, le SDK Flutter nettoie quand même la session locale
```

13.2 Activation du code de jeu

API-06 — Activation d'un code physique

Champ	Valeur
Méthode + Endpoint	POST /functions/v1/activate-code
Auth requis	OUI — Bearer JWT utilisateur
Type	Supabase Edge Function (Deno)

```
// REQUÊTE
{
  "code_value": "TRLG-4X9K-2M7P",          // code imprimé dans la boîte
  "user_id":    "550e8400-e29b-41d4-...",    // uid de l'utilisateur connecté
  "device_id":  "android-a1b2c3d4e5f6"      // device_info_plus identifier
}

// RÉPONSE 200 — Activation réussie
{
  "status":       "activated",
  "message":      "Code activé avec succès",
  "activated_at": "2025-09-15T14:32:00Z"
}

// RÉPONSE 200 — Code déjà activé sur le même appareil (reconnexion)
{
  "status":       "already_activated_same_device",
  "message":      "Reconnexion autorisée"
}

// RÉPONSE 403 — Code déjà activé sur un autre appareil
{
  "status":  "device_conflict",
  "message": "Ce code est déjà utilisé sur un autre appareil"
}

// RÉPONSE 404 — Code introuvable
{
  "status":  "not_found",
  "message": "Code invalide"
}
```

```
// RÉPONSE 422 — Corps de requête malformé
{
  "status":  "invalid_payload",
  "message": "Les champs code_value, user_id et device_id sont requis"
}
```

13.3 Profil utilisateur

API-07 — Lecture du profil

```
// GET /rest/v1/user_profiles?id=eq.<uid>&select=*
// SDK Flutter : supabase.from("user_profiles").select().eq("id", uid).single()

// RÉPONSE 200 — Profil trouvé
{
  "id":                "550e8400-...",
  "username":          "LionMaster",
  "avatar_url":        "https://...supabase.co/.../avatars/550e8400.webp",
  "total_score":       4250,
  "current_level":     7,
  "lives":             3,
  "lives_last_refill": "2025-09-15T14:00:00Z",
  "created_at":        "2025-09-01T08:00:00Z"
}

// RÉPONSE 406 — Profil introuvable (ne devrait pas arriver si création auto en
Phase 1)
{}    // Corps vide avec header Content-Range: */0
```

API-08 — Création automatique du profil (première connexion)

```
// POST /rest/v1/user_profiles
// Déclenché automatiquement par le client Flutter si profil absent

// REQUÊTE
{
  "id":           "550e8400-...",    // uid Supabase Auth
  "username":     "Joueur_a1b2",     // username par défaut (modifiable plus tard)
  "total_score":  0,
  "current_level":1,
  "lives":        5
}

// RÉPONSE 201 — Profil créé
{
  "id":            "550e8400-...",
  "username":      "Joueur_a1b2",
```

```
  "total_score":   0,
  "current_level": 1,
  "lives":         5,
  "lives_last_refill": "2025-09-15T14:32:00Z",
  "created_at":    "2025-09-15T14:32:00Z"
}

// RÉPONSE 409 — Profil déjà existant (conflit clé primaire)
{
  "code":    "23505",
  "message": "duplicate key value violates unique constraint"
}
// → Traitement Flutter : ignorer ce 409, relire le profil existant
```

13.4 Cartes et trios

API-09 — Récupération d'un trio aléatoire pour générer une question

Champ	Valeur
Méthode + Endpoint	POST /functions/v1/generate-question
Auth requis	OUI
Type	Edge Function — logique serveur pour éviter les triche clients

```
// REQUÊTE
{
  "level":          7,          // niveau actuel du joueur
  "distance_target": 2,          // D1 | D2 | D3 selon le niveau
  "config":         "B",         // A | B | C — quelle carte masquer
  "exclude_ids":    [            // IDs des questions déjà posées cette session
    "trio-id-001", "trio-id-002"
  ]
}

// RÉPONSE 200 — Question générée
{
  "question_id":   "q-550e8400-...",   // ID unique de cette question (pour logs)
  "config":        "B",
  "visible_cards": [
    {
      "id":          "card-emettrice-001",
      "card_type":   "emettrice",
      "image_path":  "emettrices/savane/lion_base.webp",
      "image_url":
"https://...supabase.co/.../emettrices/savane/lion_base.webp",
      "theme_tags":  ["animal", "lion", "savane"]
    },
    {
      "id":          "card-receptrice-001",
      "card_type":   "receptrice",
```

```
      "image_path":  "receptrices/savane/d1/lion_miroir_h.webp",
      "image_url":
"https://...supabase.co/.../receptrices/savane/d1/lion_miroir_h.webp",
      "theme_tags":  ["animal", "lion", "miroir"]
    }
  ],
  "choices": [                   // 10 cartes mélangées (1 correcte + 9
distracteurs)
    {
      "id":          "card-cable-001",
      "card_type":   "cable",
      "image_path":  "cables/geometrique/miroir_h.webp",
      "image_url":
"https://...supabase.co/.../cables/geometrique/miroir_h.webp",
      "cable_category": "geometrique"
    },
    // ... 9 autres cartes câble (distracteurs)
  ],
  "correct_answer_id": "card-cable-001",   // ID de la bonne réponse
  "time_limit_seconds": 40,
  "trio_id":           "trio-d1-001"        // pour traçabilité
}

// RÉPONSE 404 — Pas assez de trios disponibles pour ce niveau
{
  "code":    "no_trio_available",
  "message": "Aucun trio disponible pour cette distance et ce niveau"
}
```

API-10 — Validation d'une réponse joueur

Champ	Valeur
Méthode + Endpoint	POST /functions/v1/validate-answer
Auth requis	OUI
Rôle	Valide côté serveur pour éviter toute manipulation cliente du score

```
// REQUÊTE
{
  "question_id":     "q-550e8400-...",
  "session_id":      "sess-660e9500-...",
  "selected_card_id":"card-cable-001",
  "elapsed_seconds": 12,         // temps mis pour répondre
  "time_limit":      40
}

// RÉPONSE 200 — Bonne réponse
{
  "is_correct":       true,
  "base_points":      20,
  "distance_multiplier": 1.5,    // D2 → ×1.5
  "time_ratio":       0.30,      // 12s / 40s
  "time_bonus":       1.25,      // réponse entre 25% et 50%
```

```
  "total_points":     37,        // floor(20 × 1.5 × 1.25)
  "bonuses_triggered": ["BONUS_STREAK"],
  "streak_bonus_points": 20,
  "final_points":     57,
  "new_score":        1297,
  "new_streak":       4
}

// RÉPONSE 200 — Mauvaise réponse
{
  "is_correct":         false,
  "total_points":       0,
  "maluses_triggered":  ["MALUS_WRONG"],
  "lives_lost":         1,
  "new_lives":          2,
  "penalty_points":     0,
  "new_score":          1240,
  "new_streak":         0,
  "correct_card": {
    "id":        "card-cable-001",
    "image_url": "https://...supabase.co/.../cables/geometrique/miroir_h.webp"
  }
}

// RÉPONSE 200 — Timeout (elapsed >= time_limit)
{
  "is_correct":        false,
  "maluses_triggered": ["MALUS_TIMEOUT"],
  "lives_lost":        1,
  "penalty_points":    5,
  "new_lives":         2,
  "new_score":         1235,
  "correct_card": { ... }
}

// RÉPONSE 401 — Session invalide ou expirée
{
  "code":    "invalid_session",
  "message": "Session de jeu introuvable ou expirée"
}
```

13.5 Sessions de jeu

API-11 — Démarrage d'une session de jeu

```
// POST /rest/v1/game_sessions

// REQUÊTE
{
  "user_id":       "550e8400-...",
  "level_number":  7
```

```
}

// RÉPONSE 201 — Session créée
{
  "id":           "sess-660e9500-...",
  "user_id":      "550e8400-...",
  "level_number": 7,
  "score":        0,
  "completed":    false,
  "started_at":   "2025-09-15T14:32:00Z"
}
```

API-12 — Mise à jour de la session (après chaque question)

```
// PATCH /rest/v1/game_sessions?id=eq.<session_id>

// REQUÊTE — mise à jour incrémentale après bonne réponse
{
  "score":           1297,
  "correct_answers": 4,
  "bonus_earned":    57
}

// REQUÊTE — mise à jour après mauvaise réponse
{
  "wrong_answers":   2,
  "malus_received":  1
}

// RÉPONSE 200 — Mis à jour
{
  "id":              "sess-660e9500-...",
  "score":           1297,
  "correct_answers": 4,
  "wrong_answers":   2,
  "bonus_earned":    57,
  "malus_received":  1,
  "completed":       false
}
```

API-13 — Clôture d'une session (fin de niveau ou session expirée)

```
// PATCH /rest/v1/game_sessions?id=eq.<session_id>

// REQUÊTE — niveau réussi
{
  "completed":       true,
  "ended_at":        "2025-09-15T14:47:22Z",
  "duration_seconds": 922
}
```

```
// REQUÊTE — session expirée ou niveau échoué
{
  "completed":       false,
  "ended_at":        "2025-09-15T14:47:22Z",
  "duration_seconds": 900
}

// RÉPONSE 200 — Session clôturée
{
  "id":              "sess-660e9500-...",
  "completed":       true,
  "score":           1450,
  "ended_at":        "2025-09-15T14:47:22Z",
  "duration_seconds": 922
}
```

API-14 — Mise à jour du profil après niveau réussi

```
// PATCH /rest/v1/user_profiles?id=eq.<user_id>

// REQUÊTE — progression de niveau
{
  "current_level": 8,           // level + 1
  "total_score":   5700         // total_score + score_session
}

// RÉPONSE 200
{
  "id":            "550e8400-...",
  "current_level": 8,
  "total_score":   5700,
  "lives":         3
}
```

13.6 Classement

API-15 — Récupération du leaderboard

```
// GET /rest/v1/user_profiles?select=username,avatar_url,total_score,current_level
//   &order=total_score.desc&limit=50

// RÉPONSE 200
[
  {
    "username":      "LionMaster",
    "avatar_url":    "https://...supabase.co/.../avatars/lion.webp",
```

```
    "total_score":   12400,
    "current_level": 18
  },
  {
    "username":      "TRIALpro",
    "avatar_url":    null,
    "total_score":   9870,
    "current_level": 14
  },
  // ... jusqu'à 50 entrées
]
```

13.7 Tableau récapitulatif de tous les appels backend

ID	Nom	Méthode + Endpoint	Auth	Phase
API-01	Inscription email	POST /auth/v1/signup	NON	Phase 2
API-02	Connexion email	POST /auth/v1/token?grant_type=password	NON	Phase 2
API-03	Connexion Google OAuth	Redirection OAuth (SDK Flutter)	NON	Phase 2
API-04	Renvoi email vérification	POST /auth/v1/resend	NON	Phase 2
API-05	Déconnexion	POST /auth/v1/logout	OUI	Phase 4
API-06	Activation code physique	POST /functions/v1/activate-code	OUI	Phase 3
API-07	Lecture profil	GET /rest/v1/user_profiles	OUI	Phase 1/4
API-08	Création profil	POST /rest/v1/user_profiles	OUI	Phase 1
API-09	Génération question	POST /functions/v1/generate-question	OUI	Phase 5
API-10	Validation réponse	POST /functions/v1/validate-answer	OUI	Phase 5
API-11	Démarrage session	POST /rest/v1/game_sessions	OUI	Phase 5
API-12	Mise à jour session	PATCH /rest/v1/game_sessions	OUI	Phase 5
API-13	Clôture session	PATCH /rest/v1/game_sessions (completed)	OUI	Phase 5
API-14	Progression niveau	PATCH /rest/v1/user_profiles	OUI	Phase 5
API-15	Leaderboard	GET /rest/v1/user_profiles (order total_score)	OUI	Phase 4

14. SÉCURITÉ — ROW LEVEL SECURITY (RLS) SUPABASE

Sans politiques RLS activées, tout utilisateur authentifié peut lire et modifier les données de n'importe quel autre utilisateur via l'API REST publique de Supabase. Cette section définit les règles RLS pour chaque table sensible. Les règles sont appliquées côté PostgreSQL — elles ne peuvent pas être contournées côté client.

Principe général des politiques RLS

1. RLS est activé sur toutes les tables contenant des données utilisateur.
2. Par défaut (RLS activé + aucune politique) : AUCUN accès n'est autorisé.
3. Chaque politique accorde explicitement un droit (SELECT, INSERT, UPDATE, DELETE) à une condition précise.
4. Les tables "cartes" et "trios" sont en lecture seule pour tous les utilisateurs.
5. Les Edge Functions s'exécutent avec le rôle service_role qui bypass le RLS. C'est pour cela que la validation du code et du score se fait côté serveur.

14.1 Table : user_profiles

```
-- Activer RLS
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;

-- Politique 1 : Un utilisateur ne peut lire QUE son propre profil
CREATE POLICY "user_profiles_select_own"
  ON user_profiles FOR SELECT
  USING (auth.uid() = id);

-- Politique 2 : Un utilisateur ne peut créer QUE son propre profil
CREATE POLICY "user_profiles_insert_own"
  ON user_profiles FOR INSERT
  WITH CHECK (auth.uid() = id);

-- Politique 3 : Un utilisateur ne peut modifier QUE son propre profil
-- et UNIQUEMENT les champs autorisés (username, avatar_url)
-- Les champs sensibles (total_score, current_level, lives) sont modifiés
-- UNIQUEMENT par les Edge Functions (rôle service_role)
CREATE POLICY "user_profiles_update_own_safe_fields"
  ON user_profiles FOR UPDATE
  USING (auth.uid() = id)
  WITH CHECK (auth.uid() = id);

-- Politique 4 : Le leaderboard — lecture publique des champs non sensibles
uniquement
-- On crée une VIEW dédiée pour le leaderboard (pas de RLS sur la vue, limitée aux
colonnes publiques)
CREATE VIEW leaderboard AS
  SELECT username, avatar_url, total_score, current_level
  FROM user_profiles
  ORDER BY total_score DESC
  LIMIT 100;
```

```
-- La vue est accessible à tous les utilisateurs authentifiés sans exposer les
données sensibles
```

14.2 Table : game_sessions

```
ALTER TABLE game_sessions ENABLE ROW LEVEL SECURITY;

-- Un utilisateur ne lit que SES sessions
CREATE POLICY "game_sessions_select_own"
  ON game_sessions FOR SELECT
  USING (auth.uid() = user_id);

-- Un utilisateur ne peut créer des sessions QUE pour lui-même
CREATE POLICY "game_sessions_insert_own"
  ON game_sessions FOR INSERT
  WITH CHECK (auth.uid() = user_id);

-- Mise à jour : uniquement ses propres sessions ET uniquement si non complétées
-- Le champ "score" final ne peut être mis à jour que via Edge Function
(service_role)
CREATE POLICY "game_sessions_update_own_active"
  ON game_sessions FOR UPDATE
  USING (auth.uid() = user_id AND completed = FALSE);
```

14.3 Table : activation_codes

```
ALTER TABLE activation_codes ENABLE ROW LEVEL SECURITY;

-- AUCUNE politique SELECT pour les utilisateurs : ils ne peuvent pas lire les
codes
-- La vérification se fait EXCLUSIVEMENT via Edge Function (service_role)

-- Politique minimale : un utilisateur peut vérifier si SON code est activé
-- (pour afficher le statut dans les paramètres)
CREATE POLICY "activation_codes_select_own"
  ON activation_codes FOR SELECT
  USING (auth.uid() = user_id AND is_activated = TRUE);

-- INSERT et UPDATE : interdit aux utilisateurs — Edge Function uniquement
-- Pas de politique INSERT/UPDATE = refus automatique côté client
```

14.4 Tables : cards et card_trios

```
ALTER TABLE cards      ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE card_trios ENABLE ROW LEVEL SECURITY;

-- Lecture publique pour tout utilisateur authentifié (les cartes sont du contenu
de jeu)
CREATE POLICY "cards_select_authenticated"
  ON cards FOR SELECT
  TO authenticated
  USING (is_active = TRUE);   -- seules les cartes actives sont visibles

CREATE POLICY "card_trios_select_authenticated"
  ON card_trios FOR SELECT
  TO authenticated
  USING (TRUE);

-- INSERT / UPDATE / DELETE : interdit aux utilisateurs
-- Modification des cartes : service_role uniquement (backoffice admin ou
migration SQL)
```

14.5 Résumé des droits par table et par rôle

Table	anon (non connecté)	authenticated (utilisateur)	service_role (Edge Fn / admin)
user_profiles	AUCUN	SELECT/INSERT/UPDATE sur son propre enregistrement uniquement	Tous droits
game_sessions	AUCUN	SELECT/INSERT sur les siennes, UPDATE si non completed	Tous droits
activation_codes	AUCUN	SELECT (is_activated=true ET son user_id uniquement)	Tous droits
cards	AUCUN	SELECT (is_active=true uniquement)	Tous droits
card_trios	AUCUN	SELECT uniquement	Tous droits

15. GESTION DU TOKEN JWT ET DU REFRESH TOKEN

Le token JWT Supabase expire après 1 heure. Le refresh token permet d'en obtenir un nouveau sans que l'utilisateur ait à se reconnecter. Cette section documente le cycle de vie complet des tokens et les cas limites.

15.1 Cycle de vie des tokens

Token	Durée de validité	Stockage Flutter	Usage
access_token (JWT)	1 heure	Géré automatiquement par le SDK Supabase Flutter	Envoyé dans chaque requête API en Bearer
refresh_token	7 jours (glissants)	Géré automatiquement par le SDK Supabase Flutter	Échangé contre un nouveau JWT quand celui-ci expire

Comportement automatique du SDK Supabase Flutter

Le SDK Flutter de Supabase gère le refresh de manière totalement transparente :

- Il détecte automatiquement quand le JWT approche de son expiration.
- Il appelle POST /auth/v1/token?grant_type=refresh_token en arrière-plan.
- Les requêtes en cours sont mises en attente et relancées avec le nouveau token.
- Aucun code spécifique n'est nécessaire dans l'application Flutter.

Le développeur écoute simplement supabase.auth.onAuthStateChange pour réagir aux changements de session (TOKEN_REFRESHED, SIGNED_OUT, etc.).

15.2 Flux de renouvellement automatique

```
// DART — Écoute des changements d'état d'authentification
// lib/presentation/providers/auth_provider.dart

class AuthNotifier extends StateNotifier<AuthState> {
  AuthNotifier() : super(AuthState.initial()) {
    // Écouter tous les changements de session (refresh, expiration, déconnexion)
    supabase.auth.onAuthStateChange.listen((data) {
      final event   = data.event;
      final session = data.session;

      switch (event) {
        case AuthChangeEvent.signedIn:
        case AuthChangeEvent.tokenRefreshed:
          // Token renouvelé automatiquement — aucune action UI requise
          state = AuthState.authenticated(session!.user);
          break;
```

```
        case AuthChangeEvent.signedOut:
          // Déconnexion volontaire ou refresh token expiré
          state = AuthState.unauthenticated();
          // Redirection vers écran d'authentification gérée par GoRouter
          break;

        case AuthChangeEvent.userUpdated:
          state = AuthState.authenticated(session!.user);
          break;

        default:
          break;
      }
    });
  }
}
```

15.3 Cas limite — Refresh token expiré (7 jours d'inactivité)

Scénario : l'utilisateur ne lance pas l'app pendant plus de 7 jours

1. L'utilisateur relance l'app après 8 jours d'inactivité.
2. Le SDK tente de rafraîchir le JWT avec le refresh token.
3. Supabase retourne : AuthChangeEvent.signedOut (refresh token expiré).
4. L'AuthNotifier passe en état "unauthenticated".
5. GoRouter redirige automatiquement vers l'écran d'authentification.
6. L'utilisateur se reconnecte (email/mdp ou Google).
7. Vérification du code d'activation : si device_id correspond → Phase 4 directement.

NOTE : Les données locales (score, niveau) ne sont pas perdues — elles restent en base Supabase et sont rechargées après reconnexion.

```
// Appel de renouvellement (POST /auth/v1/token?grant_type=refresh_token)
// Géré automatiquement — documenté ici pour référence

// REQUÊTE (interne SDK)
{
  "grant_type":    "refresh_token",
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2g..."
}

// RÉPONSE 200 — Nouveau JWT émis
{
  "access_token":  "eyJhbGciOiJIUzI1NiIsInR...",   // nouveau JWT, +1h
  "refresh_token": "bmV3UmVmcmVzaFRva2Vu...",      // nouveau refresh token, +7j
  "token_type":    "bearer",
  "expires_in":    3600
```

```
}

// RÉPONSE 400 — Refresh token expiré
{
  "error":             "invalid_grant",
  "error_description": "Refresh Token Not Found"
}
// → Le SDK émet AuthChangeEvent.signedOut → déconnexion propre
```

16. GALERIE DES CARTES DÉBLOQUÉES

La galerie est une fonctionnalité de collection accessible depuis le menu principal. Elle affiche toutes les cartes que le joueur a rencontrées et validées correctement au moins une fois. C'est un système de progression méta qui motive la complétion de tous les niveaux.

16.1 Modèle de données — Table : user_unlocked_cards

```
-- SUPABASE / POSTGRESQL
CREATE TABLE user_unlocked_cards (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES user_profiles(id) ON DELETE CASCADE,
  card_id      UUID NOT NULL REFERENCES cards(id),
  unlocked_at  TIMESTAMPTZ DEFAULT now(),
  unlock_count INT DEFAULT 1,   -- nombre de fois que cette carte a été
correctement identifiée
  UNIQUE (user_id, card_id)     -- une carte ne peut être débloquée qu'une seule
fois par joueur
);

CREATE INDEX idx_unlocked_user ON user_unlocked_cards(user_id);
CREATE INDEX idx_unlocked_card ON user_unlocked_cards(card_id);

-- RLS : un utilisateur ne voit que SES cartes débloquées
ALTER TABLE user_unlocked_cards ENABLE ROW LEVEL SECURITY;

CREATE POLICY "unlocked_cards_own"
  ON user_unlocked_cards FOR ALL
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);
```

16.2 Logique de déverrouillage

Une carte est déverrouillée dans la galerie dès qu'un joueur donne une bonne réponse impliquant cette carte. Toutes les cartes impliquées dans le trio sont déverrouillées (Émettrice, Câble, Réceptrice).

Règle de déverrouillage — après une bonne réponse

Quand validate-answer retourne is_correct = true :

→ Déverrouiller la carte Émettrice du trio (si pas encore déverrouillée)
→ Déverrouiller la carte Câble du trio
→ Déverrouiller la carte Réceptrice du trio

Si la carte est déjà débloquée : incrémenter unlock_count uniquement

Implémentation : INSERT ... ON CONFLICT (user_id, card_id)
DO UPDATE SET unlock_count = unlock_count + 1

```
-- Déverrouillage atomique des 3 cartes d'un trio après bonne réponse
-- Exécuté dans la Edge Function validate-answer (service_role)

INSERT INTO user_unlocked_cards (user_id, card_id)
VALUES
  (<user_id>, <emettrice_id>),
  (<user_id>, <cable_id>),
  (<user_id>, <receptrice_id>)
ON CONFLICT (user_id, card_id)
DO UPDATE SET unlock_count = user_unlocked_cards.unlock_count + 1;
```

16.3 API — Lecture de la galerie

```
// GET /rest/v1/user_unlocked_cards
//   ?user_id=eq.<uid>
//
&select=card_id,unlock_count,unlocked_at,cards(id,card_type,image_path,theme_tags)
//   &order=unlocked_at.desc

// RÉPONSE 200
[
  {
    "card_id":      "card-cable-001",
    "unlock_count": 3,
    "unlocked_at":  "2025-09-15T14:32:00Z",
    "cards": {
      "id":         "card-cable-001",
      "card_type":  "cable",
      "image_path": "cables/geometrique/miroir_h.webp",
      "image_url":  "https://...supabase.co/.../cables/geometrique/miroir_h.webp",
      "theme_tags": ["miroir", "symetrie"]
    }
  },
  // ... autres cartes déverrouillées
]

// Statistiques galerie (requête agrégée)
// GET /rest/v1/user_unlocked_cards?user_id=eq.<uid>&select=count
// Retourne le nombre total de cartes débloquées
// À comparer avec SELECT count(*) FROM cards WHERE is_active=true
// pour afficher "X / Y cartes découvertes"
```

16.4 Interface galerie — organisation

Onglet galerie	Filtre appliqué	Tri par défaut

Toutes	Aucun filtre	Date de déverrouillage (récent en premier)
Émettrices	card_type = "emettrice"	Thème alphabétique
Câbles	card_type = "cable"	Catégorie (géométrique, couleur, etc.)
Réceptrices	card_type = "receptrice"	Distance puis thème
Non découvertes	Cartes actives non dans unlocked	Distance croissante (indice de recherche)

17. GESTION DES IMAGES CORROMPUES OU ABSENTES

Puisque le jeu repose entièrement sur des images, une image manquante ou corrompue peut bloquer une partie ou dégrader l'expérience. Cette section définit la politique de fallback à trois niveaux.

17.1 Les trois niveaux de défaillance image

Niveau	Cause	Impact	Fréquence attendue
Niveau 1 — Image lente	Réseau lent, latence CDN	Affichage du placeholder de chargement	Fréquent en mauvais réseau
Niveau 2 — Image absente (404)	Fichier supprimé du Storage ou image_path incorrect	Widget errorWidget affiché	Rare si convention de nommage respectée
Niveau 3 — Image corrompue	Fichier présent mais données corrompues	Erreur de décodage → errorWidget	Très rare

17.2 Politique de fallback Flutter

```
// lib/presentation/widgets/card_image_widget.dart — version avec fallback complet

CachedNetworkImage(
  imageUrl: card.imageUrl,
  fit: BoxFit.cover,

  // NIVEAU 1 : image en cours de chargement
  placeholder: (context, url) => Container(
    color: Colors.grey[200],
    child: const Center(
      child: SizedBox(
        width: 32, height: 32,
        child: CircularProgressIndicator(strokeWidth: 2),
      ),
    ),
  ),

  // NIVEAU 2 & 3 : image absente ou corrompue
  errorWidget: (context, url, error) {
    // Signalement automatique de l'erreur (non-bloquant)
    _reportBrokenImage(card.id, url, error.toString());

    // Afficher une image de remplacement générique selon le card_type
    return Container(
      color: Colors.grey[300],
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(_fallbackIcon(card.cardType), size: 40, color: Colors.grey[600]),
          const SizedBox(height: 4),
```

```
          Text(_fallbackLabel(card.cardType),
            style: TextStyle(fontSize: 10, color: Colors.grey[600])),
        ],
      ),
    );
  },
)

// Icône de secours selon le type de carte
IconData _fallbackIcon(CardType type) => switch (type) {
  CardType.emettrice  => Icons.image_outlined,
  CardType.cable      => Icons.compare_arrows,
  CardType.receptrice => Icons.auto_awesome,
};
```

17.3 Signalement automatique des images brisées

Quand une image ne charge pas, le client Flutter envoie silencieusement un rapport au backend. Un administrateur peut ensuite corriger le chemin ou ré-uploader le fichier manquant.

```
-- TABLE DE SIGNALEMENT (Supabase)
CREATE TABLE broken_image_reports (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  card_id     UUID REFERENCES cards(id),
  image_url   TEXT NOT NULL,
  error_msg   TEXT,
  reported_by UUID REFERENCES user_profiles(id),
  reported_at TIMESTAMPTZ DEFAULT now(),
  resolved    BOOLEAN DEFAULT FALSE
);

-- RLS : INSERT autorisé pour tous les users authentifiés, SELECT interdit
ALTER TABLE broken_image_reports ENABLE ROW LEVEL SECURITY;
CREATE POLICY "broken_images_insert"
  ON broken_image_reports FOR INSERT
  TO authenticated
  WITH CHECK (auth.uid() = reported_by);
```

```
// DART — Signalement non-bloquant
Future<void> _reportBrokenImage(
  String cardId, String url, String error,
) async {
  try {
    await supabase.from('broken_image_reports').insert({
      'card_id':     cardId,
      'image_url':   url,
      'error_msg':   error,
      'reported_by': supabase.auth.currentUser!.id,
    });
  } catch (_) {
    // Silencieux : ne jamais bloquer le jeu pour un rapport
  }
```

```
}
```

17.4 Règle de validation avant activation d'une carte

Pour éviter les images brisées en production, toute nouvelle carte doit passer une vérification avant que is_active soit mis à true.

Checklist de validation d'une carte avant activation

1. Le fichier existe bien dans Supabase Storage au chemin image_path
2. L'URL publique retourne un HTTP 200 (pas de 404 ni 403)
3. Le fichier est décodable comme image (format webp/png/jpg valide)
4. Les dimensions correspondent aux specs (recommandé : 512×512 px minimum)
5. La taille du fichier est < 500 Ko (optimisation mobile)
6. Le trio associé est bien enregistré dans card_trios
7. is_active peut alors passer à TRUE

18. NOTIFICATIONS PUSH

Les notifications push servent à réengager les joueurs au moment où c'est pertinent : recharge des vies, relance après inactivité, annonce de nouvelles cartes. Elles sont implémentées via Firebase Cloud Messaging (FCM) intégré à Supabase via une Edge Function dédiée.

18.1 Architecture des notifications

Composant	Rôle	Technologie
FCM (Firebase Cloud Messaging)	Service de livraison des notifications iOS et Android	Firebase
Supabase pg_cron	Planificateur de tâches PostgreSQL — déclenche les vérifications régulières	pg_cron extension
Edge Function notify-users	Identifie les utilisateurs à notifier et appelle l'API FCM	Deno / TypeScript
Table fcm_tokens	Stocke le token FCM de chaque appareil par utilisateur	Supabase PostgreSQL
Flutter firebase_messaging	Réceptionne et affiche les notifications dans l'app	flutter package

18.2 Table : fcm_tokens

```
CREATE TABLE fcm_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES user_profiles(id) ON DELETE CASCADE,
  device_id   TEXT NOT NULL,
  fcm_token   TEXT NOT NULL,
  platform    TEXT CHECK (platform IN ('android', 'ios')),
  updated_at  TIMESTAMPTZ DEFAULT now(),
  UNIQUE (user_id, device_id)
);

ALTER TABLE fcm_tokens ENABLE ROW LEVEL SECURITY;

-- Un utilisateur peut enregistrer/mettre à jour son propre token
CREATE POLICY "fcm_tokens_own"
  ON fcm_tokens FOR ALL
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);
```

18.3 Catalogue des notifications

Notification	Déclencheur	Délai	Message type	Fréquence max
Vies rechargées	lives passe de 0 à 1 (pg_cron)	Immédiat	"Tes vies sont rechargées ! Reprends la partie."	1/30 min
Vies pleines	lives = 5 (pg_cron)	Immédiat	"Toutes tes vies sont rechargées, fonce jouer !"	1/session
Inactivité 24h	Aucune session depuis 24h (pg_cron)	24h après dernière session	"Tu nous manques ! Un niveau t'attend."	1/24h
Inactivité 72h	Aucune session depuis 72h	72h	"Tes cartes t'attendent ! Reprends là où tu t'es arrêté."	1/72h
Nouvelles cartes	INSERT cards avec is_active=true (trigger)	Immédiat	"De nouvelles cartes sont disponibles !"	1/batch
Niveau débloqué	current_level incrémenté	Immédiat	"Félicitations ! Le niveau X est débloqué."	1/niveau

18.4 Enregistrement du token FCM côté Flutter

```
// lib/core/services/notification_service.dart

class NotificationService {
  static Future<void> initialize() async {
    // Demander la permission (iOS obligatoire, Android 13+)
    await FirebaseMessaging.instance.requestPermission(
      alert: true, badge: true, sound: true,
    );

    // Récupérer le token FCM de l'appareil
    final token = await FirebaseMessaging.instance.getToken();
    if (token != null) await _saveTokenToSupabase(token);

    // Écouter les renouvellements de token
    FirebaseMessaging.instance.onTokenRefresh.listen(_saveTokenToSupabase);

    // Écouter les notifications reçues en foreground
    FirebaseMessaging.onMessage.listen(_handleForegroundMessage);
  }

  static Future<void> _saveTokenToSupabase(String token) async {
    final user     = supabase.auth.currentUser;
    final deviceId = await DeviceInfoService.getDeviceId();
    if (user == null) return;

    await supabase.from('fcm_tokens').upsert({
      'user_id':   user.id,
      'device_id': deviceId,
      'fcm_token': token,
      'platform':  Platform.isAndroid ? 'android' : 'ios',
      'updated_at': DateTime.now().toIso8601String(),
    }, onConflict: 'user_id,device_id');
```

```
    }
  }
```

19. CHANGEMENT D'APPAREIL ET TRANSFERT DE CODE

Un joueur peut légitimement changer de téléphone (panne, upgrade). La règle "1 code = 1 appareil" est protectrice mais ne doit pas piéger un utilisateur de bonne foi. Cette section définit la procédure officielle de transfert.

19.1 Cas déclencheur

Scénario : le joueur possède un nouveau téléphone

1. Le joueur installe l'application sur son nouveau téléphone.
2. Il se connecte avec son compte (email/mdp ou Google).
3. Il arrive sur l'écran de saisie du code.
4. Il saisit son code d'activation (le même que sur l'ancien téléphone).
5. La Edge Function activate-code détecte : device_id différent de celui enregistré.
6. Réponse 403 "device_conflict" — accès refusé normalement.

→ Sans procédure de transfert, l'utilisateur est bloqué.
→ Solution : Edge Function transfer-device + vérification d'identité.

19.2 Edge Function : transfer-device

```
// POST /functions/v1/transfer-device
// Auth requis : OUI (Bearer JWT)

// REQUÊTE
{
  "code_value":      "TRLG-4X9K-2M7P",
  "new_device_id":   "android-newdevice-xyz789",
  "confirmation":    true    // L'utilisateur a confirmé vouloir transférer
}

// Logique côté serveur :
// 1. Vérifier que le code appartient bien à l'utilisateur authentifié (user_id =
auth.uid())
// 2. Vérifier que l'email du compte est bien vérifié (email_confirmed_at != null)
// 3. Mettre à jour device_id dans activation_codes
// 4. Invalider tous les tokens FCM de l'ancien appareil
// 5. Logger le transfert dans device_transfer_logs

// RÉPONSE 200 — Transfert autorisé
{
  "status":       "transferred",
  "message":      "Appareil mis à jour avec succès",
  "transferred_at": "2025-09-15T14:32:00Z"
}
```

```
// RÉPONSE 403 — Code n'appartient pas à cet utilisateur
{
  "status":  "unauthorized",
  "message": "Ce code n'est pas associé à votre compte"
}

// RÉPONSE 429 — Trop de transferts (anti-abus : max 3 transferts par code)
{
  "status":  "transfer_limit_reached",
  "message": "Nombre maximum de transferts atteint. Contactez le support."
}
```

19.3 Table : device_transfer_logs

```
CREATE TABLE device_transfer_logs (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code_value      TEXT NOT NULL,
  user_id         UUID REFERENCES user_profiles(id),
  old_device_id   TEXT NOT NULL,
  new_device_id   TEXT NOT NULL,
  transferred_at  TIMESTAMPTZ DEFAULT now(),
  ip_address      TEXT    -- pour audit de sécurité
);

-- Compter les transferts pour la limite anti-abus
CREATE INDEX idx_transfer_code ON device_transfer_logs(code_value);

-- Règle : maximum 3 transferts autorisés par code
-- Vérifié dans la Edge Function transfer-device :
SELECT count(*) FROM device_transfer_logs WHERE code_value = $1;
-- Si count >= 3 → retourner 429
```

19.4 Expérience utilisateur lors du changement d'appareil

Étape	Action utilisateur	Comportement app
1	Saisit le code sur le nouveau téléphone	Réponse 403 device_conflict détectée
2	Affichage automatique modal "Nouveau téléphone ?"	Modal avec explication et bouton "Transférer vers cet appareil"
3	Appuie sur "Transférer vers cet appareil"	Appel POST /functions/v1/transfer-device
4	Confirmation de transfert	Réponse 200 → activation normale → Menu principal
5 (échec)	Limite de 3 transferts atteinte	Message "Contactez le support à support@trialgo.com"

20. PROCESSUS D'AJOUT DE NOUVELLES CARTES EN PRODUCTION

Cette section documente le workflow complet pour qu'un nouveau lot de cartes dessinées par le chef artistique arrive dans l'application mobile des joueurs, sans erreur ni interruption de service.

20.1 Vue d'ensemble du pipeline de publication

Pipeline en 5 étapes — du dessin à la production

ÉTAPE 1 — Création artistique
- Chef artistique dessine les cartes (Emettrice + Câble + Réceptrice)
- Format : .webp, 512×512 px, < 500 Ko

ÉTAPE 2 — Upload dans Supabase Storage
- Upload dans le bucket trialgo-cards selon la convention de nommage
- is_active = FALSE par défaut à l'insertion

ÉTAPE 3 — Insertion en base de données
- INSERT dans la table cards (is_active = FALSE)
- INSERT dans card_trios pour lier les 3 cartes

ÉTAPE 4 — Validation technique
- Vérification checklist (URL accessible, format valide, dimensions, taille)
- Test is_coherent() sur le trio nouvellement créé

ÉTAPE 5 — Activation
- UPDATE cards SET is_active = TRUE WHERE id IN (...)
- Notification push "Nouvelles cartes disponibles !" (si lot >= 5 cartes)

20.2 Détail de chaque étape avec scripts

Étape 1 — Nommage et préparation des fichiers

Checklist pré-upload pour le chef artistique

- ☐ Fichier en format .webp (convertir via Squoosh ou cwebp si nécessaire)
- ☐ Dimensions exactes : 512×512 pixels
- ☐ Taille fichier : < 500 Ko
- ☐ Nom de fichier respecte la convention (minuscules, tirets bas, pas d'accents)
- ☐ Pour une Réceptrice D2 : le nom contient les 2 transformations dans l'ordre

- ☐ Trio complet prêt : les 3 fichiers (E + C + R) sont disponibles en même temps

Étape 2 & 3 — Upload et insertion en base

```
-- Script SQL à exécuter dans l'éditeur SQL Supabase (ou via migration)
-- Exemple : ajout d'un nouveau trio D1 "Éléphant + Miroir V"

-- 1. Insérer la carte Émettrice (si nouvelle)
INSERT INTO cards (card_type, distance_level, image_path, image_width,
image_height,
                   theme_tags, is_active)
VALUES ('emettrice', 1, 'emettrices/savane/elephant_base.webp', 512, 512,
        ARRAY['animal','elephant','savane'], FALSE)
RETURNING id;  -- noter <id_elephant_base>

-- 2. Le Câble "miroir_v" existe déjà → récupérer son ID
SELECT id FROM cards
WHERE image_path = 'cables/geometrique/miroir_v.webp';
-- noter <id_cable_miroir_v>

-- 3. Insérer la Réceptrice D1
INSERT INTO cards (card_type, distance_level, image_path, image_width,
image_height,
                   theme_tags, parent_emettrice_id, parent_cable_id, is_active)
VALUES ('receptrice', 1, 'receptrices/savane/d1/elephant_miroir_v.webp', 512, 512,
        ARRAY['animal','elephant','miroir'],
        <id_elephant_base>, <id_cable_miroir_v>, FALSE)
RETURNING id;  -- noter <id_elephant_miroir_v>

-- 4. Enregistrer le trio
INSERT INTO card_trios (emettrice_id, cable_id, receptrice_id, distance_level)
VALUES (<id_elephant_base>, <id_cable_miroir_v>, <id_elephant_miroir_v>, 1);
```

Étape 4 — Script de validation automatique

```
-- Vérification du trio nouvellement créé

-- Test 1 : les 3 images sont bien référencées
SELECT c.id, c.card_type, c.image_path, c.is_active
FROM card_trios ct
JOIN cards c ON c.id IN (ct.emettrice_id, ct.cable_id, ct.receptrice_id)
WHERE ct.id = <id_trio_nouveau>;

-- Test 2 : la relation de cohérence est valide
SELECT is_coherent(<id_elephant_base>, <id_cable_miroir_v>,
<id_elephant_miroir_v>);
-- Doit retourner TRUE

-- Test 3 : aucune carte du trio n'est déjà active par erreur
SELECT id, image_path, is_active FROM cards
```

```
WHERE id IN (<id_elephant_base>, <id_cable_miroir_v>, <id_elephant_miroir_v>)
  AND is_active = TRUE;
-- Doit retourner 0 lignes
```

Étape 5 — Activation en production

```
-- Activation d'un lot de cartes après validation complète
UPDATE cards
SET is_active = TRUE
WHERE id IN (
  <id_elephant_base>,
  <id_elephant_miroir_v>
  -- NB : le câble miroir_v était déjà actif, pas besoin de le réactiver
);

-- Vérification post-activation
SELECT id, card_type, image_path, is_active
FROM cards
WHERE id IN (<id_elephant_base>, <id_cable_miroir_v>, <id_elephant_miroir_v>);
-- Toutes les lignes doivent avoir is_active = TRUE

-- Si >= 5 nouvelles cartes activées dans ce lot : déclencher notification push
-- POST /functions/v1/notify-users { "type": "new_cards", "count": 3 }
```

20.3 Rôles et responsabilités

Étape	Responsable	Outil utilisé	Validation par
Création des illustrations	Chef artistique	Logiciel de dessin (Procreate, Illustrator...)	Chef de projet
Conversion en .webp	Chef artistique ou Chef technique	Squoosh.app ou script cwebp	Chef technique
Upload Supabase Storage	Chef technique	Dashboard Supabase Storage ou Supabase CLI	Chef technique
INSERT SQL en base	Chef technique	Éditeur SQL Supabase Dashboard	Chef technique
Validation technique	Chef technique	Script SQL de validation (Section 20.2)	Chef de projet
Activation is_active=TRUE	Chef de projet	Éditeur SQL Supabase Dashboard	Chef de projet

21. RÉCAPITULATIF COMPLET — TOUS LES CAS DE FIGURE

Cas de figure	Mécanique déclenchée	Résultat
Upload nouvelle carte	Chef artistique → Storage → INSERT cards (is_active=FALSE)	Carte invisible dans le jeu jusqu'à validation et activation
Activation lot de cartes	UPDATE is_active=TRUE + Edge Function notify-users	Cartes jouables + notification push aux joueurs
Image absente ou corrompue (404)	CachedNetworkImage → errorWidget + _reportBrokenImage	Icône de fallback + rapport automatique dans broken_image_reports
Inscription email	POST /auth/v1/signup + email vérification	Compte créé → écran activation code
Connexion Google	OAuth Google via Supabase + Deep Link	Session créée → vérification code
JWT expiré (1h)	SDK Flutter → POST /auth/v1/token?grant_type=refresh_token	Nouveau JWT transparent, aucune action utilisateur
Refresh token expiré (7j)	SDK → AuthChangeEvent.signedOut	Déconnexion propre → écran auth
Code valide non activé	Edge Function activate-code → Device ID enregistré	Accès au jeu déverrouillé
Code déjà activé même appareil	Edge Function → device_id identique → 200 already_activated	Reconnexion autorisée
Code déjà activé autre appareil	Edge Function → 403 device_conflict	Modal "Nouveau téléphone ?" affiché
Transfert vers nouveau téléphone	Edge Function transfer-device → UPDATE device_id	Transfert autorisé (max 3 fois)
Transfert > 3 fois	Edge Function → 429 transfer_limit_reached	Redirection vers support@trialgo.com
Config A : trouver Réceptrice	GenerateQuestion D1 config A	Question facile, niveaux 1-5
Config B : trouver Câble-image	GenerateQuestion config B	Question intermédiaire, niveaux 4-10
Config C : trouver Émettrice	GenerateQuestion config C	Question difficile, niveaux 8+
Bonne réponse	validate-answer → TRUE + BonusService	Points calculés + 3 cartes déverrouillées dans galerie
Mauvaise réponse	validate-answer → FALSE + MalusService	-1 vie, bonne image révélée 1.5s
Chrono de tour expiré	MALUS_TIMEOUT	-1 vie + -5 pts
Réponse < 25% du temps	BONUS_TURBO	Score × 1.5
3 bonnes consécutives	BONUS_STREAK	+20 pts/réponse suivante
Niveau sans faute	BONUS_PERFECT	Score niveau × 2
Config C réussie	BONUS_EXPERT	+30 pts + badge
Vies = 0	Session terminée, score sauvegardé	Écran "Plus de vies" + timer recharge
30 min écoulées, vies < 5	pg_cron → refill_lives() + notification FCM	+1 vie + notification "Vies rechargées"
Inactivité 24h	pg_cron → Edge Function notify-users	Notification push de relance
Niveau réussi	UPDATE user_profiles current_level++ + notification	Niveau suivant débloqué + notification

Session > durée max	MALUS_SESSION_TIMEOUT	Score actuel sauvegardé, session fermée
Bonne réponse (tout niveau)	INSERT user_unlocked_cards (3 cartes du trio)	Cartes ajoutées à la galerie
Consultation galerie	GET user_unlocked_cards JOIN cards	Grille d'images avec filtre par type
Tentative lecture données autre utilisateur	RLS PostgreSQL → refus automatique	HTTP 403 — aucune donnée exposée
Accès cartes inactives	RLS cards → is_active=FALSE filtré	Cartes invisibles pour les utilisateurs
Signalement image brisée (auto)	INSERT broken_image_reports	Rapport disponible pour admin dans dashboard
Nouvelle carte uploadée + validée	UPDATE is_active=TRUE	Disponible immédiatement pour tous les joueurs

TRIALGO — Recueil technique complet v5.0. *Ce document couvre l'intégralité du projet : principe de jeu, modèles de données, Storage, génération de cartes, mécanique solo, niveaux, bonus/malus, workflow utilisateur, contrat API (15 endpoints), sécurité RLS, gestion des tokens JWT, galerie, images corrompues, notifications push, transfert d'appareil et processus de publication des cartes en production.*